



DEPARTMENT OF COMPUTER SCIENCE

An Evaluation and Multi-Pass Extension of the Emek-Rosén Semi-Streaming Set Cover Algorithm

Michael Barlow

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree
of Master of Engineering in the Faculty of Engineering.

Friday 15th May, 2020

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MEng in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the author. This project did not require ethical review, as determined by Dr Christian Konrad, the supervisor of the author.

Michael Barlow, Friday 15th May, 2020

Contents

1	Introduction	1
1.1	Streaming Algorithms	3
1.1.1	Streaming Set Cover	3
1.2	Related Work	4
1.3	Motivation and Objectives	4
2	Preliminaries	7
2.1	The SET COVER Problem	7
2.1.1	Example	7
2.1.2	Hypergraph Equivalent	7
2.1.3	Set Cover for Graphs	7
2.2	Approximation Algorithms	8
2.3	Data Access Models	8
2.3.1	Offline Algorithms	8
2.3.2	Streaming Algorithms	9
2.3.3	Semi-Streaming Algorithms	9
2.4	Greedy Set Cover	10
3	Semi-Streaming Set Cover	13
3.1	Disk Friendly Greedy	13
3.2	Emek-Rosén Semi-Streaming Set Cover	14
3.3	Implementation	16
3.3.1	Language Considerations	16
3.3.2	Common Structure	17
3.3.3	Implementation of DFG	17
3.3.4	Implementation of SSSC	18
3.4	Benchmark Data Sets	20
3.4.1	Data Set Statistics	20
3.5	Empirical Analysis	20
3.6	Approximation Factor	22
3.6.1	Accounting for Disproportionality	24
4	Multiple Passes	27
4.1	Generalising SSSC to Multiple Passes	27
4.2	Progressive Greedy	31
4.2.1	Approximation Factor	32
4.3	Implementation	33
4.3.1	Implementation of MSSSC	33
4.3.2	Implementation of Progressive Greedy	34
4.4	Empirical Analysis	34
5	Conclusion	37
5.1	Contributions	37
5.2	Future Work	37
5.2.1	Other Streaming Algorithms	38
5.2.2	SSSC at Web Scale	38
5.2.3	Modification to SSSC	38

A Set Size Distribution	43
B Multi-Pass Solution Quality	45

List of Figures

1.1	An instance of the SET COVER problem, visualised as a hypergraph.	1
1.2	Direct access to remote data is a bottleneck for algorithms when processing massive data sets.	2
1.3	A comparison of permitted access patterns. Streams can only be accessed in sequence. . .	3
1.4	Counter values after traversing $\mathcal{A}$	3
2.1	An example instance of the SET COVER problem.	7
2.2	Equivalent representations of a directed graph.	8
2.3	An example set stream.	9
2.4	An adversarial instance of the SET COVER problem, shown here as a hypergraph, where the input comprises entirely of singleton sets.	9
2.5	A graph encoded as a stream of edges.	9
3.1	The cover certificate χ returned by SSSC when given as input the example problem definition described in Section 2.1.1. Here, $\text{Im}(\chi) = \{\text{id}(S_1), \text{id}(S_2)\}$	15
3.2	Effective subset example.	15
3.3	Binary encoding of a set S in disk-based DFG, where $x_i \in S$ for each $1 \leq i \leq S $	17
3.4	An example effectiveness frequency distribution.	19
4.1	A comparison between the contribution distributions given by SSSC and DFG for the file <code>twitter_graph.dat</code>	27
4.2	Universe decomposition for p passes of MSSSC.	28
4.3	Solution size comparison as pass count varies for data file <code>twitter_graph.dat</code>	34
4.4	Contribution distribution given by 12-pass MSSSC for the file <code>twitter_graph.dat</code>	35
A.1	Set size distribution for the file <code>twitter_graph.dat</code>	43

List of Tables

3.1	Main benchmark data set contexts.	20
3.2	Benchmark data set statistics.	21
3.3	Specification of system used for experiments.	21
3.4	A direct performance comparison between disk-based DFG ($p = 1.065$) and SSSC.	22
B.1	Solution sizes given by p -pass MSSSC as p increases in powers of 2.	45
B.2	Solution sizes given by p -pass Progressive Greedy as p increases in powers of 2.	46

List of Algorithms

2.1	Greedy	10
3.1	Disk Friendly Greedy	14
3.2	Semi-Streaming Set Cover	16
3.3	Compute Effective Subset (Sort)	16
3.4	Compute Effective Subset (Count)	19
4.1	Multi-pass Semi Streaming Set Cover	31
4.2	Naïve p -pass Progressive Greedy	32
4.3	p -pass Progressive Greedy	32

Abstract

The combinatorial optimisation problem of SET COVER is to find a minimal subcollection of sets which together contain every element in some universe. This NP-hard problem arises at the heart of many analysis tasks, especially in the domain of connected data. Under the widely-believed conjecture that $P \neq NP$, it becomes increasingly infeasible to exactly solve SET COVER as input sizes grow, so we instead turn our attention to approximating an optimal solution, which can be done in polynomial time. The widely-used greedy algorithm is known to be essentially optimal from both a theoretical and a practical perspective. However, this algorithm has a linear space requirement and accesses memory arbitrarily, so it scales poorly to large data sets. With this in mind, we specifically consider the SET COVER problem under the semi-streaming model, where, for an instance with n elements and m sets, space usage is limited to $O(n \text{ polylog}(n, m))$ bits and the sets that comprise the problem instance arrive one at a time. Semi-Streaming Set Cover, an algorithm proposed in a purely theoretical setting by Emek and Rosén, makes a single pass over the input. The main aim of this work is to determine whether this algorithm provides a competitive alternative to the greedy algorithm—or variants thereof—in a practical setting, when problem instances are very large.

Our main result is a strongly positive answer to this question, which we complement with a theoretical analysis of (our specialised version of) the algorithm in terms of the largest set size Δ . We also generalise the core workings to a new algorithm which makes p passes over the input data. We show that our new algorithm, which we call Multi-pass Semi-Streaming Set Cover, processes each set in linear time, and produces an $O((p + 1)\Delta^{1/(p+1)})$ -approximate solution. With an empirical comparison to an alternative algorithm which uses a greedy heuristic and executes under the same model, we demonstrate that our algorithm is highly capable, especially when few passes over the input are made.

Supporting Technologies

- The GNU C++ compiler (<https://gcc.gnu.org/>) facilitated the compilation of our implementations.
- Python was used alongside matplotlib (<https://matplotlib.org/>) to generate graphical plots.
- Wolfram|Alpha (<https://www.wolframalpha.com/>) was used to check mathematical results.

Notation and Acronyms

$\mathbb{R}$	The set of all real numbers
$\mathbb{Q}$	The set of all rational numbers
$\mathbb{Z}$	The set of all integers
$\mathcal{S}$	The set collection for a given SET COVER instance
$\mathcal{U}$	The universe for a given SET COVER instance
OPT	The optimal solution for a given SET COVER instance
n	The universe size for a given SET COVER instance
m	The number of sets in a given SET COVER instance
Δ	The maximum set size for a given SET COVER instance
SSSC	Semi-Streaming Set Cover
MSSSC	Multi-pass Semi-Streaming Set Cover
DFG	Disk Friendly Greedy
RAM	Random Access Memory
XOR	Exclusive OR

Acknowledgements

First of all, I would like to thank my supervisor Dr Christian Konrad for the invaluable support and guidance he has given me throughout the course of this work. Christian's enthusiasm and expertise has been a great inspiration to me, and has developed my interest in algorithms and their theoretical analysis. I also owe a great debt of gratitude to my friends. In particular, I thank Sam, with whom I had many great discussions, and Dan, with whom I consumed a great deal of tea. Finally, I thank my family for their love and support.

Chapter 1

Introduction

Given a collection of sets containing elements from some universe, the SET COVER problem is to find a minimal subcollection of these sets such that the union over this subcollection equals the universe. This fundamental problem in computer science is known to be NP-hard [35], and features either directly or in obscured form in a wide range of applications.

Alongside the traditional set collection definition, the SET COVER problem can be equivalently expressed in terms of hypergraphs, which are networks of vertices connected by hyperedges where each hyperedge connects one or more vertices. The problem now is to determine a minimal collection of hyperedges which together are incident to all vertices in the hypergraph. We demonstrate both the SET COVER problem and this equivalence in Figure 1.1. Here, the subcollection $\{S_1, S_3, S_4\}$ contains all elements in the universe, however the minimal subcollection containing all elements, and hence the optimal solution to the SET COVER problem in this instance, is $\{S_1, S_2\}$.

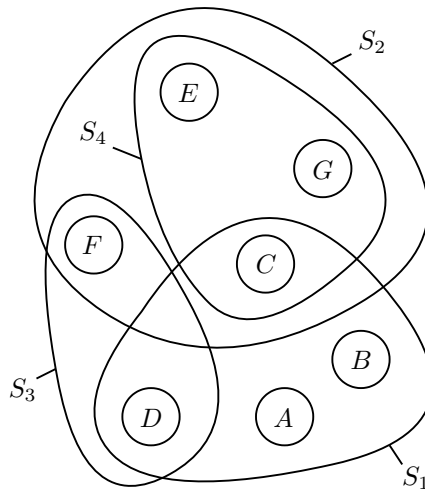


Figure 1.1: An instance of the SET COVER problem, visualised as a hypergraph.

Noting that all graphs can be interpreted as hypergraphs, the potential domain of applications for the SET COVER problem is vast. To demonstrate this, we now list some specific examples of settings where an instance of the SET COVER problem arises.

Social Networks. In social networks, it is often desired to extract a set of influencers, i.e., a minimal set of social network users who can together directly contact all other users of the network.¹ By aggregating connections for each user into sets, this can be seen as an instance of SET COVER. We can also consider social network communities (or groups): if the membership list of each community is known, a minimal set of communities which together contain all network users can be found.

Code Coverage. Software (especially in safety-critical systems) is often developed alongside a suite of tests which flag incorrect or unexpected behaviour. Exhaustively applying this test suite to the

¹This is exactly the DOMINATING SET problem, which is equivalent to SET COVER under linear reductions [33].

software can have a high cost, both in terms of resource usage and time. For this reason, such testing sessions are typically only performed prior to production releases. To test software changes during development in a less thorough but cheaper manner, one could find and apply a minimal collection of test cases which together cause the execution of every line in the code base. This leads to an instance of SET COVER, where the lines of source code comprise the universe and each test case becomes a set containing the lines of code which it executes [53, 13, 64].

Information Retrieval. Saha and Getoor [58] describe the SET COVER problem in the context of blogs. We can consider all online blogs as a hypergraph, where vertices correspond to topics, and each hyperedge contains all topics relevant to a single blog. Then, for a given set of interesting topics, a minimal (and so not overwhelming) collection of relevant blogs can be found. This use case generalises to the searching and querying of document stores of any kind, as demonstrated by Anagnostopoulos et al. [4].

Geographical Coverage. There are many ways to encode geographical data in hypergraph form. We consider an example where vertices correspond to houses, and hyperedges correspond to the service range of facilities. By solving the SET COVER problem in this context, we can determine a minimal collection of facilities which could together serve all houses. Such geographical insight may be valuable if, for instance, one was deploying a service in a new location, or if an established service requires downscaling. Indeed, for some similar use cases, a geometric variant of the SET COVER problem may be more appropriate (see [14, 51, 2, 5] and the references therein).

Miscellaneous. Other example applications include web crawling [60], tableau generation [27], feature selection [17, 9], track linking [61], broadcasting in wireless ad hoc networks [55, 44, 45], query optimisation [50], protein interaction prediction [32], and interference reduction [37] to name a few.

The NP-hardness of SET COVER means it quickly becomes infeasible to exactly solve the problem as input sizes grow. Often, however, an exact solution is not necessary. In these cases, we can instead find an approximate solution which guarantees to be close to optimal; this can be done in polynomial time. The best-known approximation algorithm for the SET COVER problem uses a simple greedy heuristic: at each iteration, the algorithm finds the set containing the most as-yet-uncovered elements, and adds it to the solution. This greedy algorithm guarantees a solution whose size is within a factor $\ln n - \ln \ln n + \Theta(1)$ of the optimal solution for a universe of size n [59]; in terms of approximation ratios this is essentially as good as it gets, as the SET COVER problem cannot be approximated to within a factor of $(1 - o(1)) \ln n$ unless $P = NP$ [21].

Many algorithms, such as the greedy one described, assume direct access to data, i.e., the time taken to fetch an arbitrary data point is constant. In most cases, this assumption is feasible: modern computers are typically fitted with at least 16 gigabytes of random access memory (RAM), allowing very quick access to stored values. However, when input data sizes far-exceed the capacity of a computers' RAM, direct access can be incredibly costly. In these scenarios the data is stored elsewhere, such as on disk² or over a network, meaning regular arbitrary data source queries quickly form the bottleneck shown in Figure 1.2. As the global data volume is ever-growing [54], this problem is particularly relevant to the present day. The *streaming* computation model is a means to alleviate the data access bottleneck, and, with this in mind, we consider the SET COVER problem in both the direct access and streaming contexts.

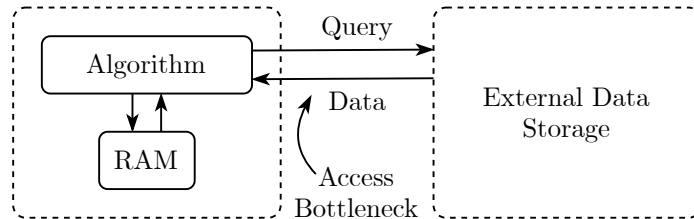


Figure 1.2: Direct access to remote data is a bottleneck for algorithms when processing massive data sets.

²We use *disk* as a general term to refer to the class of storage devices secondary to RAM.

1.1 Streaming Algorithms

Streaming is a model for data access which imposes two primary restrictions to facilitate processing at a large scale. Namely, as shown in Figure 1.3, algorithms under the streaming model may only access data sequentially in a so-called *stream*, and such algorithms are only permitted access to space sublinear in input size. This limit on space ensures algorithms under this model cannot just read the whole input into RAM and directly access it from there; rather, some small summary of the data must be stored from which conclusions can eventually be drawn. The suitability of this model of computation extends to sources which naturally take the form of a stream, such as data generated in real-time.

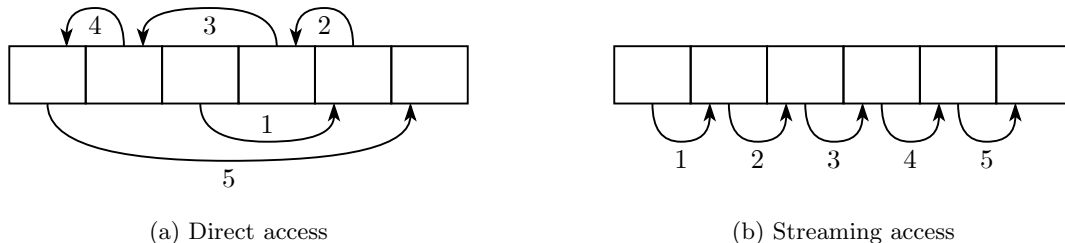


Figure 1.3: A comparison of permitted access patterns. Streams can only be accessed in sequence.

The key goal of algorithms under the streaming model is to minimise the size of working memory, and, if possible, make only a single pass over the input. We consider a streaming algorithm to be highly space efficient if the number of bits it requires is polylogarithmic in input size. To illustrate this, consider the following example.

Problem 1 (Locating the single unique value in a stream). Let $\mathcal{A} = \{A_1, A_2, \dots, A_m\}$ be an input stream of m positive integers, where all but one of the elements appear exactly twice. Let n denote the largest integer in $\mathcal{A}$. Which element appears only once?

As an example instance of this problem, consider the input $\mathcal{A} = \{5, 7, 3, 3, 5, 14, 7\}$. Each element arrives one at a time in the order presented, i.e., 5 arrives first, followed by 7, followed by 3, and so on. Requiring linear space, a naïve solution would be to maintain a counter bit for each integer in the range 1 to n ; upon arrival of an element, the value of the corresponding bit is flipped. Applying this approach to the example instance results in the 14-bit vector shown in Figure 1.4, and the solution to the problem is then simply the value whose counter bit is set.

Value	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Counter bit	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Figure 1.4: Counter values after traversing $\mathcal{A}$.

While the correctness of this approach is clear, it requires exactly n bits, which quickly becomes impractical as the problem size grows.

We now consider an alternative solution, which exploits two key properties of the bitwise XOR ($\oplus$) operator. For this solution, a variable a is introduced and initialised to 0. If we simply XOR the binary representation of each element with a as it arrives in the input stream, a takes on the final value $A_1 \oplus A_2 \oplus \dots \oplus A_m$. We know that (1) XOR is an associative operator; and (2) the XOR of a value with itself is 0, so this final value is equivalent to the binary representation of the element which only appears once. For the example instance,

$$a = A_1 \oplus A_2 \oplus \dots \oplus A_m = 0101_2 \oplus 0111_2 \oplus 0011_2 \oplus 0011_2 \oplus 0101_2 \oplus 1110_2 \oplus 0111_2 = 1110_2 = 14_{10},$$

as expected. The number of bits required for this approach exactly equals the number of bits required to represent the largest value in the input stream, and, as all values in the input stream are at most n , no more than $\lceil \log n \rceil$ bits of working memory are used. Indeed, this approach is optimal, as in the worst case exactly $\lceil \log n \rceil$ bits are needed just to store the solution.

1.1.1 Streaming Set Cover

Hereafter, we denote the universe size of an instance of SET COVER by n , and the number of sets by m .

Solving graph—and, by extension, hypergraph—problems in polylogarithmic space is often impractical, and, in the case of SET COVER, not always possible. We realise this simply by noting that a resulting solution would be of size $O(n \log m)$ bits which is sublinear—but not polylogarithmic—to the size of the problem instance. For this reason, we consider the SET COVER problem under the *semi-streaming* model of computation: a variant of the streaming model where (hyper)edges arrive in the input stream, and a slightly increased (but still sublinear) memory usage is considered highly space efficient. The semi-streaming model was initially formalised by Feigenbaum et al. [24] for graph problems, and was later extended by Emek and Rosén to hypergraph problems such as SET COVER [22]. For hypergraph problems, an algorithm under the semi-streaming model is permitted access to $O(n \text{ polylog}(n, m))$ bits of space.

1.2 Related Work

Cormode et al. [16] introduce the Disk Friendly Greedy (DFG) algorithm, which attempts to reduce the impact of the access bottleneck seen in the original greedy algorithm by processing data in sizeable contiguous chunks. Despite the fact that DFG mostly uses space external to RAM (indeed, RAM usage obeys semi-streaming constraints), the total space usage is still linear in size of input. Due to the mostly sequential access pattern, DFG scales very well to larger data sets; Cormode et al. demonstrate experimentally that the solution sizes produced by DFG very closely match (and in some cases improve upon) those given by the aforementioned greedy algorithm, and are (for large problems) obtained in a fraction of the time.

Considering arrival of data in a stream, Alon et al. [3] first explore this space by introducing a version of the problem where all sets (and hence universe elements) are known by the algorithm from the outset. The input to this problem takes the form of a stream of elements, where, after receiving an element, an algorithm must produce a set which contains it. The goal then is to have used as few sets as possible upon termination of the stream. This, however, is not the streaming model that we focus on. Instead, we consider a different streaming format: the sets (rather than the elements) of the SET COVER instance arrive sequentially. This setting requires no prior knowledge of the problem instance, and was first described by Saha and Getoor [58], who proceeded to give a sublinear-space $O(\log n)$ -approximation algorithm which makes $O(\log^2 n)$ passes over the input set stream. Since then, other sublinear-space algorithms have been described under this set arrival model [52, 20, 30, 10, 8], though we now shift our attention specifically to semi-streaming algorithms.

Emek and Rosén [22] explore the limitations of approximating SET COVER in just a single pass with a semi-streaming space constraint. They introduce Semi-Streaming Set Cover (SSSC), a one-pass algorithm for a slight variant of SET COVER problem we describe, which obtains an $O(\sqrt{n})$ -approximate solution to our version of the problem. Chakrabarti and Wirth [12] also consider this semi-streaming model. They introduce Progressive Greedy, an algorithm which is somewhat similar in approach to DFG, applying a greedy heuristic in p passes over the input stream, and producing a $(p + 1)n^{1/(p+1)}$ -approximate solution. Assadi and Khanna [7] introduce an $O(\log n \cdot \log m)$ -pass semi-streaming algorithm which works on *dynamic* streams (those which also permit insertion and deletion of sets), and gives an $O(\log n)$ -approximate solution.

Considering theoretical bounds, Demaine et al. [20] show that no deterministic streaming algorithm can achieve a constant-factor approximation to the SET COVER problem in space sublinear to input size. Chakrabarti and Wirth complement this result by showing that no (deterministic or randomised) p -pass semi-streaming algorithm can give an approximation factor better than $0.99n^{1/(p+1)}/(p + 1)^2$. Assadi et al. [8] further discuss the space complexity of the streaming SET COVER problem in a single-pass setting, and Assadi [6] extends this discussion to the multi-pass setting.

1.3 Motivation and Objectives

The work we present has three key objectives, which we now motivate and describe.

SSSC in Practice. Though significant work has been done to establish the theoretical guarantees (and limitations) of algorithms and computational models relevant to the SET COVER problem, there are comparatively few works which consider such algorithms from a practical standpoint, especially when data sets are large. Grossman and Wool [29] demonstrate that the aforementioned greedy algorithm performs very well in practice by testing it on small SET COVER instances where exact

solutions are known. They also note that the theoretical upper bound does not well predict the practical results. In a later study, Gomes et al. [28] also form this conclusion, finding that the output of the greedy algorithm deviated on average only 7% from the optimal solution over 60 (small) SET COVER instances. Coremode et al. [16] consider larger data sets, showing that DFG produces solution sizes comparable those given by the greedy algorithm. However, as we mention previously, DFG requires access to (slow, and potentially costly) external memory. If we can achieve similar solution sizes to those given by DFG (and thus in turn the greedy algorithm, which we know is essentially optimal from a theoretical perspective) without the external memory access overhead, then this would clearly be an important result. To this end, we consider SSSC, which was described in a purely theoretical context by Emek and Rosén [22]. At the cost of a comparatively weak approximation factor, desirable characteristics of SSSC include (1) semi-streaming space usage; (2) polynomial time processing of each set with simple logic; and (3) single-pass processing of the incident stream. Indeed, such properties are especially valuable for the processing of very large data sets, where problem instances may otherwise be intractable. With this in mind, the first objective of this work is to determine whether SSSC performs well in practice by empirically comparing it with DFG.

Enhancing Approximation. Algorithms for the SET COVER problem typically express their approximation guarantees in terms of either n or m . We note, however, that SET COVER data sets have a third important parameter: the size of the largest set, which we denote Δ . If a data set has a very large universe, but all sets are reasonably small, then an approximation factor in terms of Δ would be more relevant. Indeed, the value of Δ can never exceed n , so an approximation factor in terms of Δ is more valuable. The second objective of this work is to express the approximation guarantees of SSSC in terms of Δ .

Generalising SSSC. As originally given by Emek and Rosén, SSSC presents no clear way of improving the solution it produces (short of implementing a different algorithm). The final objective of this work is to generalise SSSC to an algorithm which makes p -passes over the input stream. We aim to provide the approximation guarantees of this generalised algorithm, alongside an empirical evaluation.

Each of these three objectives encapsulate a number of lower-level objectives, which we now detail in the natural order of completion.

- Modify SSSC such that it approximates the version of SET COVER addressed in this project (un-weighted SET COVER).
- Implement both DFG and SSSC in a language suitable for performance comparison. Include in these implementations code to measure solution size, execution time, peak RAM usage, and, in the case of DFG, bytes read from and written to disk.
- Collate a suite of large benchmark data sets. These data sets should share a common format, with variations in size and structure.
- Empirically compare SSSC with DFG using this data set suite. Explain the results and draw conclusions accordingly.
- Extend the theoretical analysis of SSSC by giving an approximation factor in terms of the size of the largest input set.
- Determine how multiple passes could be used to improve the solution quality of SSSC, and thus generalise SSSC to a p -pass algorithm.
- Extend the theoretical analysis of SSSC to this generalised algorithm.
- Empirically compare this generalised version of SSSC to an appropriate alternative algorithm which operates under the same computational model. Establish the strengths and weaknesses of both.

Chapter 2

Preliminaries

This chapter introduces relevant definitions and corresponding mathematical notation to be used throughout.

2.1 The Set Cover Problem

Problem 2 (SET COVER). The SET COVER problem comprises two components: a universe $\mathcal{U}$ with cardinality $|\mathcal{U}| = n$, and a collection $\mathcal{S}$ of m subsets of $\mathcal{U}$, i.e., $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ where $S_i \subseteq \mathcal{U}$ for all $1 \leq i \leq m$. The collection $\mathcal{S}$ satisfies $\bigcup_{S \in \mathcal{S}} S = \mathcal{U}$, and the size of the largest set is denoted by Δ . Some subcollection $\mathcal{S}'$ of $\mathcal{S}$ is said to *cover* $\mathcal{U}$ if for all $x \in \mathcal{U}$, there exists some set $S \in \mathcal{S}'$ containing x . The aim of the SET COVER problem is to find a cover of minimal size, referred to hereafter as OPT.

2.1.1 Example

Figure 2.1 gives a formal definition of the problem instance shown in Figure 1.1. Here, $n = 7$, $m = 4$, $\Delta = 4$ and the minimal cover is $\text{OPT} = \{S_1, S_2\}$.

$\mathcal{U}$	$\{A, B, C, D, E, F, G\}$	S_1	$\{A, B, C, D\}$
$\mathcal{S}$	$\{S_1, S_2, S_3, S_4\}$	S_2	$\{C, E, F, G\}$
		S_3	$\{D, F\}$
		S_4	$\{C, E, G\}$

Figure 2.1: An example instance of the SET COVER problem.

2.1.2 Hypergraph Equivalent

A *graph* $G = (V, E)$ is a two-tuple of sets: V contains the vertices of the graph, and E contains the edges. An edge is a pair of vertices, where $(u, v) \in E$ denotes an edge from u to v , with $u, v \in V$. A *hypergraph* is a natural extension of G . In a hypergraph, the edge pair constraint is removed; instead, each edge is replaced with a *hyperedge* $e \in E$, which is an arbitrary non-empty subset $e \subseteq V$. It is hereafter assumed that V contains no isolated vertices, i.e., all vertices are incident to at least one hyperedge. Subsequently, the SET COVER problem as described in Problem 2 can be expressed as a hypergraph, with V equivalent to the universe $\mathcal{U}$, and E equivalent to the set collection $\mathcal{S}$. The aim is now to find a minimal collection of hyperedges which are incident to all vertices in the hypergraph.

As the definitions of the traditional SET COVER problem and the hypergraph version differ only in notation, the former is preferred hereafter, although hypergraphs may be used for the purposes of visualisation.

2.1.3 Set Cover for Graphs

To consider how the SET COVER problem applies to graphs, we first give some definitions.

Definition 2.1.1 (EDGE COVER). An EDGE COVER of a graph is a subset $\mathcal{E} \subseteq E$ such that every vertex $v \in V$ is incident to at least one edge $e \in \mathcal{E}$.

Definition 2.1.2 (DOMINATING SET). A DOMINATING SET for a graph G is a subset $\mathcal{D} \subseteq V$ such that for every vertex $v \in V \setminus \mathcal{D}$ there exists some vertex $u \in \mathcal{D}$ which is adjacent to v (with u leading to v in the case that G is directed).

As hypergraphs are a generalisation of graphs, the SET COVER problem directly applies to a graph in the usual way (i.e., treat E as the set collection $\mathcal{S}$ and V as the universe $\mathcal{U}$), where each set is of size two. The resulting solution corresponds to a minimal EDGE COVER of the graph. However, if we instead work with the vertex-centric *adjacency list* graph representation, the solution given by SET COVER has an altogether different meaning. As shown in Figure 2.2, an adjacency list is a two-column table which gives the neighbours of each vertex in an unordered set. If the vertex corresponding to each row is included in this unordered set and we solve SET COVER for the resulting list, the solution then is a minimal DOMINATING SET [33]. Indeed, though we do not discuss it further here, this concept generalises to hypergraphs (see [1] for the relevant definitions).

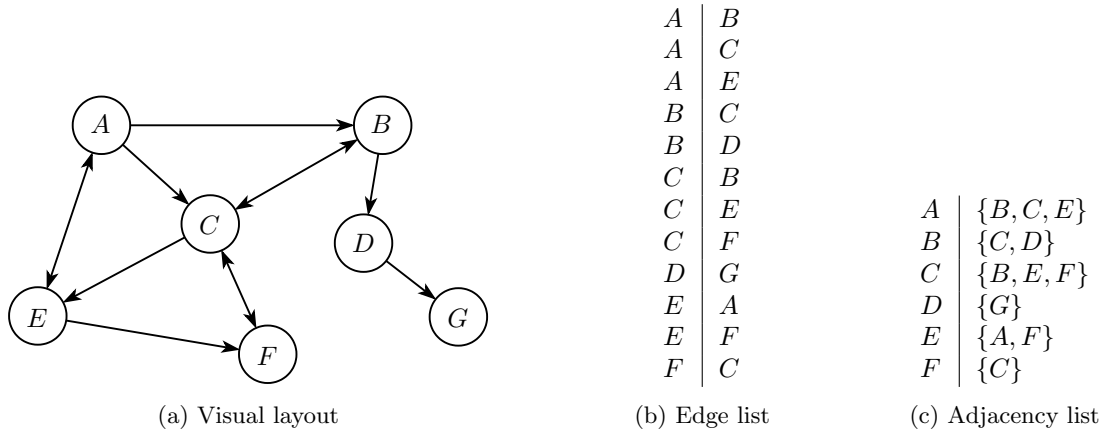


Figure 2.2: Equivalent representations of a directed graph.

2.2 Approximation Algorithms

Finding an exact value OPT for an instance of the SET COVER problem is NP-hard, and hence asymptotically infeasible under the widely-believed conjecture that $P \neq NP$. Instead, it often suffices to find a cover approximating OPT, that is, find a cover with cardinality sufficiently close to |OPT| to still fulfil the desired purpose. This is advantageous as such an approximation can be found in polynomial time.

By introducing an α -factor, we can put a fixed bound on the difference in solution size between OPT and that returned by an approximation algorithm. Specifically, SET COVER is a minimisation problem so an α -approximation algorithm produces a solution at most an α -factor larger than the size of OPT. In the context of the SET COVER problem, an α -approximation algorithm provides a cover $\mathcal{S}'$ where $|\text{OPT}| \leq |\mathcal{S}'| \leq \alpha \cdot |\text{OPT}|$ for some $\alpha > 1$.

2.3 Data Access Models

2.3.1 Offline Algorithms

An *offline* algorithm is one which receives the whole input in one go, and is permitted access to enough memory (be it local or external) to store this entire input. Such an algorithm thus has *direct access* to the resulting data structure, i.e., the algorithm is able to query the input arbitrarily. In the context of the SET COVER problem, an offline algorithm is able to maintain a full copy of the set collection $\mathcal{S}$.

If the input fully resides in RAM, the time taken for direct access queries is typically small and constant (or close thereto). However, if the input resides on disk, direct accesses are orders of magnitude slower. This disk overhead can be somewhat reduced if the access patterns of the algorithm are predictable: if *sequential locality* is observed by accessing data in large linear chunks, then the algorithm can make effective use of the cache.

2.3.2 Streaming Algorithms

As illustrated by the comparison with direct access in Figure 1.3, a *stream* is a sequence of data points which arrive as input to an algorithm one at a time. Since the size of this stream is often unknown (size is typically only discovered upon arrival of the final data point) or very large, a complete history of the stream cannot necessarily be stored in memory. For this reason, algorithms under the streaming model, that is, *streaming algorithms*, maintain some internal summary of the visited portion of the input in space sublinear—and ideally polylogarithmic—to the size of the stream. As the whole input does not reside in memory, a second pass over the stream would be required to access data points which have already been visited; additional passes such as this are permitted in the context of *multi-pass* algorithms. In contrast, a *one-pass* algorithm is constrained to a single traversal of the stream.

Considering the SET COVER problem under the streaming model, the set collection $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ forms the input stream, with the set S_t arriving at time t . Figure 2.3 illustrates an example of this, where the problem encoded in the stream is that given in Section 2.1.1.

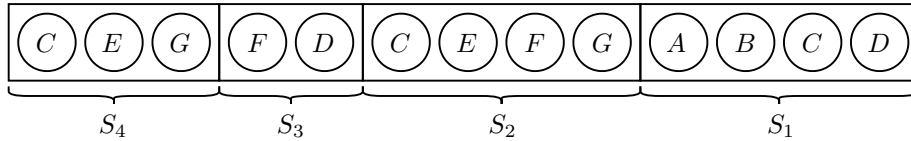


Figure 2.3: An example set stream.

In contrast to the offline context, in which direct access allows unique identification of sets by their index i , indices are not applicable for set streams as the elements cannot be accessed arbitrarily. Instead, under the streaming model, a set could be identified by, for example, the time t at which it first arrives. So as to abstract this implementation detail and generalise notation for both classes of algorithm, it is assumed that every set $S \in \mathcal{S}$ is uniquely associated with an *identifier* $\text{id}(S)$ of size $O(\log m)$ bits. After receiving in some form the set collection $\mathcal{S}$ as input, an algorithm for the SET COVER problem subsequently outputs a cover of $\mathcal{S}$ expressed as a collection of identifiers $\mathcal{I}$. When considering an adversarial case where there are m input singleton sets such as that depicted in Figure 2.4, it is clear that space sublinear to m is insufficient. In this case, $|\text{OPT}| = n = m$, with each set $S \in \mathcal{S}'$ requiring $O(\log m)$ bits. This means $O(n \log m)$ bits are needed to simply store $\mathcal{I}$, the resulting cover of $\mathcal{S}$. For this reason, it is not possible to approximate every instance of the SET COVER problem in space polylogarithmic to size of input.

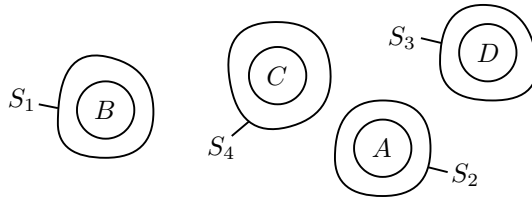


Figure 2.4: An adversarial instance of the SET COVER problem, shown here as a hypergraph, where the input comprises entirely of singleton sets.

2.3.3 Semi-Streaming Algorithms

Formalised by Feigenbaum et al. [24] for the context of graph processing, *semi-streaming* algorithms are those which receive as input an n -vertex graph encoded as a stream of edges (see Figure 2.5), and are permitted $O(n \text{ polylog } n)$ bits of space. Emek and Rosén [22] extend this definition to hypergraph inputs with m hyperedges by allowing $O(n \text{ polylog}(n, m))$ bits of space; this accounts for the inability to uniquely identify a hyperedge with $O(\log n)$ bits.

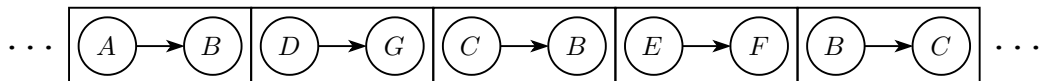


Figure 2.5: A graph encoded as a stream of edges.

Considering the SET COVER problem under this model, where again $\mathcal{S}$ forms the input stream with the set S_t arriving at time t , space is now clearly sufficient to store a cover $\mathcal{I}$ of $\mathcal{S}$, which requires $O(n \log m)$ bits. Note, however, that the semi-streaming space allowance is still far smaller than the $\Theta(nm)$ space required to fully represent an instance of the SET COVER problem, so algorithms under this model cannot simply store a full copy of the input stream in memory.

2.4 Greedy Set Cover

A *greedy* algorithm is one which attempts to find an optimal solution by making locally optimal choices at each execution step. Such a greedy heuristic is demonstrated in a simple and well-known offline approach to the SET COVER problem, described fully in Algorithm 2.1. Intuitively, by maintaining a collection C of elements covered so far, this algorithm selects the set from $\mathcal{S}$ containing the most uncovered elements at each execution step, adding this set to the solution and updating C . This process is repeated until all elements in the universe are covered. As proven by Slavík [59], this algorithm produces a solution whose size is within a factor $(\ln n - \ln \ln n + 0.78)$ of $|\text{OPT}|$, which is not far from optimal: Dinur and Steurer [21] extend a classic result of Feige [23] to show that approximating the SET COVER problem to within a factor $(1 - \epsilon) \ln n$ of $|\text{OPT}|$ is NP-hard for every $\epsilon > 0$. Slavík also gives a lower bound, showing that for every $n \geq 2$ there exists some instance of SET COVER where the greedy algorithm outputs a solution with size greater than $(\ln n - \ln \ln n - 0.31) \cdot |\text{OPT}|$.

Algorithm 2.1: Greedy

input : A collection of sets $\mathcal{S}$
output: A collection of identifiers which together cover $\mathcal{S}$

```

1  $\mathcal{I}, C \leftarrow \emptyset$ 
2 while  $C \neq \mathcal{U}$  do
3    $i \leftarrow i'$  which maximises  $|S_{i'} \setminus C|$ 
4    $\mathcal{I} \leftarrow \mathcal{I} \cup \{\text{id}(S_i)\}$ 
5    $C \leftarrow C \cup S_i$ 
6 end
7 return  $\mathcal{I}$ 
```

From an implementation perspective, the primary difficulty with the greedy algorithm is determining which set contains the most uncovered elements. At each loop iteration, new elements are included C which might also appear in other sets. As a result, some mechanism is required to track which sets contain those elements freshly included in C , such that the respective uncovered element counts of those sets may be reduced. The two prevailing solutions to this problem are briefly described.¹

Multiple Passes. A simple approach is to calculate $|S \setminus C|$ for every $\{S \in \mathcal{S} \mid \text{id}(S) \notin \mathcal{I}\}$ at each execution step, adding the set with the maximal such value to the solution. Though this method has the benefit that memory accesses are sequential, a pass over most of the input is required each time a set is added to the solution. The time complexity is thus $O(|\mathcal{I}| \cdot \sum_{S \in \mathcal{S}} |S|)$, meaning the algorithm will be very slow when solution sizes are large.

Inverted Index. An alternative approach is to maintain a priority queue containing the $|S_i \setminus C|$ values for each $1 \leq i \leq m$. So that this priority queue can be updated when a set is added to the solution, an inverted index is pre-computed, which stores for each $x \in \mathcal{U}$ a set $T_x = \{i \mid x \in S_i\}$. Intuitively, at each execution step, the highest priority set S_i is removed from the priority queue and added to the solution, and the $|S \setminus C|$ values indexed by T_j for each $j \in S_i \setminus C$ are subsequently updated to reflect the change in C . The time complexity of this approach is $O((\log m) \cdot \sum_{x \in \mathcal{U}} |T_x|)$, which can be equivalently expressed as $O((\log m) \cdot \sum_{S \in \mathcal{S}} |S|)$ for ease of comparison with the multi-pass method. While this solution is asymptotically more efficient in most cases (i.e., when $|\mathcal{I}| > \log m$), the frequent arbitrary memory access patterns seen both during and after the construction of the inverted index mean sequential locality is not observed and hence the algorithm may run slowly when the problem instance does not fit in RAM.

¹Naturally, other solutions exist. Stergiou and Tsioutsoulis [60] introduce and discuss some further noteworthy alternatives.

We note that this direct access greedy algorithm can be converted into a semi-streaming algorithm by slightly modifying the aforementioned multiple pass approach. With a stream as input, we can find the largest $|S \setminus C|$ value in a single pass and then add it to the solution. Repeating this until $C = \mathcal{U}$, a solution can be found using only $O(n \log m)$ bits of space. However, the pass complexity of this approach is highly undesirable: in the worst case, n passes over the input stream would be required.

Chapter 3

Semi-Streaming Set Cover

As discussed in Section 2.4, approximating the SET COVER problem with the greedy algorithm is essentially optimal: a significant improvement to the approximation factor cannot be achieved by an algorithm that runs in polynomial time. However, especially for large data sets, the logic of this greedy algorithm proves awkward to implement. Cormode et al. [16] introduce the Disk Friendly Greedy algorithm, showing that a slight relaxation of the greedy heuristic enables a more practical and better-scaling implementation without significantly impacting theoretical guarantees. Despite the strong performance, Disk Friendly Greedy still requires access to the whole input throughout execution, and, for very large data sets, this can become a bottleneck. We wish to determine whether Semi-Streaming Set Cover, a simpler non-greedy one-pass algorithm introduced by Emek and Rosén [22], provides a feasible alternative for large problems when external memory access is expensive. In this chapter, we aim to answer this question both empirically and theoretically. To this end, we first formally introduce both algorithms, then we discuss their implementation, after which we evaluate them with a suite of data sets.

3.1 Disk Friendly Greedy

Proposed by Cormode et al. [16], the offline Disk Friendly Greedy (DFG) algorithm aims to provide a practical means of solving the SET COVER problem for data sets which exceed the capacity of RAM. In these cases, the asymptotically favourable implementation of the traditional greedy algorithm described in Section 2.4 is inefficient due to the unpredictable disk accesses it makes throughout execution. Recognising this bottleneck, Cormode et al. ensure that DFG has a mostly sequential disk access pattern. As mentioned previously, the key difficulty with the greedy algorithm is determining which set contains the most as-yet-uncovered elements. Rather than finding such a set, DFG approximates this step, settling with a set whose uncovered element count is close to maximal. To achieve this, DFG introduces an input parameter $p \in \mathbb{R}_{>1}$ which influences both the running time and approximation factor of the algorithm. Initially, the input collection $\mathcal{S}$ is partitioned into subcollections, where set $S \in \mathcal{S}$ is assigned to subcollection $\mathcal{S}^{(k)}$ if $p^k \leq |S| < p^{k+1}$. The appropriate subcollection $\mathcal{S}^{(k)}$ can be thus found for a given set $S \in \mathcal{S}$ in constant time by evaluating

$$k = \left\lfloor \frac{\log |S|}{\log p} \right\rfloor.$$

Starting with the subcollection of largest k (denoted $\mathcal{S}^{(K)}$), the algorithm considers each set in turn, adding the set to the solution if the number of uncovered elements it contains is at least p^k . If not, the already-covered elements are removed from the set, and it is subsequently reassigned to subcollection $\mathcal{S}^{(k')}$, where $k' < k$. The algorithm repeats this process for each $1 \leq k < K$ in descending order, after which each set $S \in \mathcal{S}^{(0)}$ is added to the solution if $|S \setminus C| = 1$. This logic admits efficient cache usage: subcollections are processed one at a time, and the sets within each subcollection are processed sequentially. If stored contiguously in memory (such as in a file), then sequential locality is observed when processing a subcollection.

Formally described in Algorithm 3.1, DFG has a time complexity of $O((1 + 1/(p - 1)) \cdot \sum_{S \in \mathcal{S}} |S|)$ and produces a solution which has size within a factor $1 + p \ln n$ of $|\text{OPT}|$. In practice, Cormode et al. demonstrate that the solutions provided by DFG are typically very close in size to those given by the greedy algorithm. Furthermore, they show that DFG can process medium and large sized inputs in a fraction of the time it takes the greedy algorithm to do the same.

Algorithm 3.1: Disk Friendly Greedy

input : A collection of sets $\mathcal{S}$, and a value $p > 1$
output: A collection of identifiers which together cover $\mathcal{S}$

```

1  $\mathcal{I}, C \leftarrow \emptyset$ 
2  $K \leftarrow 0$ 
   /* Initial partition of input sets into subcollections */
3 for  $S_i \in \mathcal{S}$  do
4    $k \leftarrow \lfloor (\log |S_i|) / (\log p) \rfloor$ 
5    $K \leftarrow \max(K, k)$ 
6    $\mathcal{S}^{(k)} \leftarrow \mathcal{S}^{(k)} \cup \{S_i\}$ 
7 end
   /* Process subcollections  $\mathcal{S}^{(K)}$  to  $\mathcal{S}^{(1)}$  */
8 for  $k \leftarrow K$  to 1 do
9   for  $S_i \in \mathcal{S}^{(k)}$  do
10    if  $|S_i \setminus C| \geq p^k$  then
11       $\mathcal{I} \leftarrow \mathcal{I} \cup \{\text{id}(S_i)\}$ 
12       $C \leftarrow C \cup S_i$ 
13    else
14       $S_i \leftarrow S_i \setminus C$ 
15       $k' \leftarrow \lfloor (\log |S_i|) / (\log p) \rfloor$ 
16       $\mathcal{S}^{(k')} \leftarrow \mathcal{S}^{(k')} \cup \{S_i\}$ 
17    end
18  end
19 end
   /* Process subcollection  $\mathcal{S}^{(0)}$  */
20 for  $S_i \in \mathcal{S}^{(0)}$  do
21   if  $|S_i \setminus C| = 1$  then
22      $\mathcal{I} \leftarrow \mathcal{I} \cup \{\text{id}(S_i)\}$ 
23      $C \leftarrow C \cup S_i$ 
24   end
25 end
26 return  $\mathcal{I}$ 

```

3.2 Emek-Rosén Semi-Streaming Set Cover

Semi-Streaming SET COVER (SSSC), proposed by Emek and Rosén [22], is a one-pass algorithm which addresses the *weighted* SET COVER problem. An extension to the SET COVER problem described in Problem 2, the weighted counterpart introduces element *benefits* and set *costs*, denoted $b(x) \in \mathbb{Q}_{>0}$ and $c(S) \in \mathbb{Q}_{>0}$ respectively for $x \in \mathcal{U}$ and $S \in \mathcal{S}$. Rather than covering all elements, a further term $0 < \delta \leq 1$ is introduced, where some subset $F \subseteq \mathcal{U}$ is said to δ -cover $\mathcal{U}$ if $\sum_{x \in F} b(x) \geq \delta \cdot \sum_{x \in \mathcal{U}} b(x)$. The goal then is to construct a subcollection $\mathcal{S}' \subseteq \mathcal{S}$ where the set of all covered elements δ -covers $\mathcal{U}$, such that the total cost $\sum_{S \in \mathcal{S}'} c(S)$ is minimised.

In the direct access memory model, given a set of identifiers $\mathcal{I}$, the input can be examined to determine which set $S \in \mathcal{S}$ covers some element $x \in \mathcal{U}$ where $\text{id}(S) \in \mathcal{I}$. As this task would require an additional pass over the stream in the (semi-)streaming context, SSSC instead returns a *cover certificate* χ , which is a partial function from $\mathcal{U}$ to $\{\text{id}(S) \mid S \in \mathcal{S}\}$ with *domain*

$$\text{Dom}(\chi) = \{x \in \mathcal{U} \mid \chi \text{ is defined over } x\}$$

and *image*

$$\text{Im}(\chi) = \{\text{id}(S) \mid \exists x \in \text{Dom}(\chi) \text{ s.t. } \chi(x) = \text{id}(S)\}$$

that satisfies (1) if $x \in \text{Dom}(\chi)$ and $\chi(x) = \text{id}(S)$, then $x \in S$; and (2) $\sum_{x \in \text{Dom}(\chi)} b(x) \geq \delta \cdot \sum_{x \in \mathcal{U}} b(x)$. The image of χ by definition consists of the identifiers of the sets included in $\mathcal{S}'$, so the output of SSSC can be expressed as a set of identifiers $\mathcal{I}$ by evaluating $\text{Im}(\chi)$.

As the focus of this work is SET COVER in the *unweighted* context, that is, SET COVER as described in Problem 2, SSSC will hereafter be considered only in simplified form where the parameters are restricted to $\delta = 1$, $b(x) = 1$ for all $x \in \mathcal{U}$, and $c(S) = 1$ for all $S \in \mathcal{S}$; introducing such constraints reduces the weighted SET COVER problem to the unweighted version. Furthermore, as shown in Figure 3.1, the returned cover certificate χ is now a total function from $\mathcal{U}$ to $\{\text{id}(S) \mid S \in \mathcal{S}\}$ with the same image, where it still holds that if $x \in \text{Dom}(\chi)$ and $\chi(x) = \text{id}(S)$, then $x \in S$.

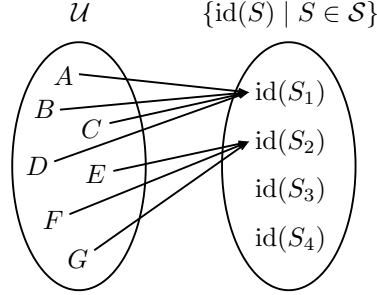


Figure 3.1: The cover certificate χ returned by SSSC when given as input the example problem definition described in Section 2.1.1. Here, $\text{Im}(\chi) = \{\text{id}(S_1), \text{id}(S_2)\}$.

SSSC maintains for each $x \in \mathcal{U}$ an integer *effectiveness* and an *effectiveness identifier*, denoted $\text{eff}(x)$ and $\text{eid}(x)$ respectively. The effectiveness of an element x aims to quantify how well x is covered by the solution thus far, with the effectiveness identifier storing $\text{id}(S)$ for the set S currently covering x . Let $\text{eff}_t(x)$ and $\text{eid}_t(x)$ denote values of $\text{eff}(x)$ and $\text{eid}(x)$ respectively for some $x \in \mathcal{U}$ at time t (i.e., just before set $S_t \in \mathcal{S}$ is processed). We now establish two central definitions.

Definition 3.2.1 (Level). The *level* of some set S is defined as

$$\text{lev}(S) = \lceil \log_2 |S| \rceil.$$

Definition 3.2.2 (Effective subset). A subset $T \subseteq S_t$ is said to be *effective* at time t if and only if all elements in T have an effectiveness strictly less than the level of T . More formally,

$$T \subseteq_{\text{eff}_t} S_t \quad \text{iff.} \quad \forall x \in T, \text{eff}_t(x) < \text{lev}(T).$$

Figure 3.2 shows an example of an effective subset. Here, $S_t = \{E, F, H, A, D, B, C, G\}$, and the effectiveness values corresponding to these elements are given. The underlined elements together form the effective subset $\{E, H, A, D, C\}$. Indeed, this is the maximal effective subset in this example: if any (or all) other elements were added to this subset, the level would not increase, and thus the subset would cease to be effective.

$x \in S_t$	<u>E</u>	F	<u>H</u>	<u>A</u>	<u>D</u>	B	<u>C</u>	G
$\text{eff}_t(x)$	2	4	1	0	2	4	-1	3

Figure 3.2: Effective subset example.

As described in Algorithm 3.2, SSSC starts by (implicitly) initialising $\text{eff}(x)$ and $\text{eid}(x)$ for every $x \in \mathcal{U}$ to NULL and -1 respectively. Then, sets are read from the stream one by one, with set S_t being read at time t . When considering set S_t , an effective subset $T \subseteq_{\text{eff}_t} S_t$ is first found with maximal cardinality, after which for each $x \in T$ the effectiveness of x is set to the level of T , and the effectiveness identifier is set to that of S_t . Intuitively, this means the set S_t is only added to the cover certificate if it contains a subset $T \subseteq S_t$ where the size of T is at least a factor of 2 greater than the size of the subsets previously designated to covering the elements in T . The algorithm proceeds in this way until the stream terminates, at which point it returns the resulting cover certificate (alongside the final effectiveness values if desired). It is worth noting that if a set is added to the cover certificate during execution, it is not guaranteed to be present in the final solution, as the elements which it is designated to cover may be included in subsequent effective subsets.

Algorithm 3.2: Semi-Streaming Set Cover

input : A stream of sets $\mathcal{S}$
output: A cover certificate of $\mathcal{S}$ alongside final effectiveness values
1 **foreach** $x \in \mathcal{U}$ **do** $\text{eid}(x) \leftarrow \text{NULL}$; $\text{eff}(x) \leftarrow -1$
2 **for** $t \leftarrow 1$ **to** m **do**
3 Read the set S_t from $\mathcal{S}$
4 $T \leftarrow T' \subseteq_{\text{eff}_t} S_t$ which maximises $|T'|$
5 **for** $x \in T$ **do**
6 $\text{eid}(x) \leftarrow \text{id}(S_t)$
7 $\text{eff}(x) \leftarrow \text{lev}(T)$
8 **end**
9 **end**
10 **return** $\text{eid}(\cdot)$ **and** $\text{eff}(\cdot)$

Observation 3.2.1. *If $T \subseteq_{\text{eff}_t} S_t$ and $x \in T$, then $T \cup \{y\}$ is effective at time t for every $y \in S_t$ such that $\text{eff}_t(y) \leq \text{eff}_t(x)$.*

Though only briefly discussed, to compute the maximal effective subset $T \subseteq S_t$, Emek and Rosén present Observation 3.2.1 and suggest sorting the elements in S_t by descending effectiveness. When subsequently iterating through the sorted list, the first encountered effective element defines the starting point from which the list tail equals T . This approach is formalised in Algorithm 3.3, and, due to the initial sort, has a time complexity of $O(|S_t| \log |S_t|)$.

Algorithm 3.3: Compute Effective Subset (Sort)

input : $S_t \in \mathcal{S}$, the set at time t
output: An effective subset $T \subseteq_{\text{eff}_t} S_t$ which maximises $|T|$
1 Sort S_t by descending $\text{eff}_t(x)$ for $x \in S_t$
2 **for** $x_i \in S_t$ **do** // S_t is indexed from 0
3 **if** $\text{lev}(|S_t| - i) > \text{eff}_t(x_i)$ **then**
4 **return** $\{x_j \in S_t \mid j \geq i\}$
5 **end**
6 **end**
7 **return** $\emptyset$

Considering the algorithm as a whole, SSSC produces a solution whose cardinality is within a factor of $\sqrt{n} + 8\sqrt{n}|\text{OPT}|$. The time complexity is $O(\sum_{S \in \mathcal{S}} |S| \log |S|)$ and the space complexity is $\Theta(n \log m)$ bits. For an algorithm which operates on streams, SSSC exhibits two beneficial properties. Firstly, from a logical perspective, no prior knowledge of the input stream is required; neither n or m need to be known in advance for the algorithm to execute. Secondly, if execution is interrupted, the stored cover certificate will always fully cover the known portion of the universe. These properties, alongside the single-pass nature of the algorithm, make SSSC an ideal choice for the processing of real-time data streams.

3.3 Implementation

To allow empirical testing, implementations of both algorithms are required. We now detail these implementations and relevant peripheral scripts.

3.3.1 Language Considerations

The key performance metric we consider is the size of the produced solution, and, if this were the only measurement taken, a high level language such as Python would be a suitable choice. However, in addition to solution size, we also wish to measure the time taken to process a data set, as well as memory usage. For this reason, it is necessary that all implementations have minimal overhead, both in terms of time and space. To achieve this, a language with ample control at the low level would be required, such as C. As some of the algorithms require dynamic arrays and hash tables—data structures not natively supported in C—we instead opted for C++, which has comparable performance while also offering these data structures in the standard library. For consistency and fairness, all algorithms were implemented in C++.

3.3.2 Common Structure

It is important that the code is clean, readable, minimal and well commented, such that the implementations can be used by others in future research. To facilitate this, both algorithms are implemented with a common structure, which decouples core workings from any user or data set input/output. In addition, both algorithms are provided alongside a `Makefile` which handles the executable build, and have a command line interface which allows for easy interaction. The resulting covers can be exported to a text file with a common format: one set identifier per line. An auxiliary C++ script exploits this common format to confirm that returned covers indeed contain all elements of the appropriate universe.

Both implementations contains code to measure performance. We measure solution size, time taken, peak RAM usage and, in the case of DFG, bytes read from and written to disk. The time taken includes all steps in the algorithm execution, including the reading of the data set from disk, but excluding the writing of the solution to file. For DFG, the initial reading of the data set is excluded from the disk read/write byte count.

3.3.3 Implementation of DFG

We implemented two versions of DFG: disk-based and RAM-based. The former is suitable for large data sets, while the latter offers a faster alternative for those data sets small enough to fully reside in RAM.

Disk-Based DFG

This version of DFG is suitable for those inputs whose size exceeds the capacity of RAM. The algorithm starts by constructing a disk-resident program friendly data structure from the original data set. This data structure comprises one binary file per subcollection, which contains all corresponding sets. Each set is encoded by a sequence of 8 byte chunks (interpreted as `uint64_t`). The first of these chunks contains the set index, and subsequent chunks contain the elements of the set, with one element per chunk. The whole sequence is then terminated with the sentinel value $2^{64} - 1$, which marks the end of the set. This encoding scheme is illustrated in Figure 3.3. To avoid the reading and writing of a sentinel chunk, an alternative method of delimiting the sets would be to use the most significant bit in the index chunk as a flag indicating the start of a new set. Note that, while using a sentinel chunk may produce incorrect results if the sentinel value features in the data set, the bit flag approach may fail if any element has the most significant bit set—effectively halving the element size capacity. For this reason, we opted for the sentinel chunk design, and confirmed that none of the benchmark data sets contained the sentinel value. There are, of course, alternative designs which altogether remove the need for delimiting in the binary file, and hence make no assumptions about element values. For example, one could also maintain a binary meta file for each subcollection: this file would contain an 8 byte entry per set, corresponding to the number of elements in that set. Though we also implemented this, it encroached on the open file limit imposed by the operating system, so we deemed it unnecessarily verbose for the purposes of benchmarking. A further alternative is to maintain this meta data in RAM, though this has the undesired effect of introducing a linear space dependency on the total number of sets in the input.

id(S)	x_1	x_2	x_3	$\dots$	$x_{ S -1}$	$x_{ S }$	$2^{64}-1$
-----------	-------	-------	-------	---------	-------------	-----------	------------

Figure 3.3: Binary encoding of a set S in disk-based DFG, where $x_i \in S$ for each $1 \leq i \leq |S|$.

To prevent the continuous opening and closing of subcollection files, a hash table (implemented with the C++ `unordered_map` data structure) maintains the pairings of k values to file streams. It is also necessary for the algorithm to know which elements have already been covered—this is implemented with the C++ `unordered_set` data structure. Together, these two data structures account for the majority of RAM usage, which is linear with respect to universe size.

RAM-Based DFG

For data sets which can be fully-contained in RAM, the use of disk (or other external memory) is unnecessary. Instead, this version of DFG maintains the full problem in a hash table, which maps each integer set index to a dynamic array containing the elements of the set. This structure was chosen in preference

to a 2D dynamic array, as it was experimentally found to be more resilient to continuously changing set sizes. The hash table, implemented with the C++ `unordered_map` data structure, allows for average constant-time set retrieval. A secondary hash table, also implemented with the C++ `unordered_map` data structure, maintains the subcollections by mapping the values of k to the identifiers of all sets in the corresponding subcollection. When the data set is first read in, both of these structures are populated, and are subsequently manipulated when a cover is extracted. As in the disk-based implementation, an `unordered_set` is used to track which elements have been covered at any given time during execution. This RAM-based DFG implementation requires space linear to the product of the universe size and the set count.

DFG Parameters

Cormode et al. [16] found in testing that smaller covers were obtained when using a threshold of p^{k-1} (rather than p^k) to determine whether to include each set in $\mathcal{S}^{(k)}$ in the final solution. Through informal experimentation, we find that such a change to the threshold typically makes little difference to the resulting solution sizes. In fact, on average, covers with the original threshold are slightly smaller. In any case, such a change modifies the approximation factor from the original value of $1 + p \ln n$ to a less favourable $1 + p^2 \ln n$ using the same analysis method. With this in mind, we implement DFG with the original threshold of p^k , and, in keeping with the experimentation by Cormode et al., we default the value of p to 1.05 for disk-based DFG and 0.001 for RAM-based DFG, though this value can be modified from the command line.

3.3.4 Implementation of SSSC

The implementation of SSSC is comparatively simple. A hash table (`unordered_map`) is maintained which maps each element to a pair of values, the first of which is the current effectiveness, and the second of which is the identifier of the set which currently covers the element. This whole structure also serves as the cover certificate output of the algorithm. This is the only data structure stored in RAM, and as such the space complexity of this implementation has a linear dependency on the size of the universe. To extract an output consistent in format with that produced by DFG, i.e., a succinct list of set identifiers which together cover the universe, all the unique identifiers in the aforementioned hash table are found using an `unordered_set`.

Finding the Maximal Effective Subset

An implementation difficulty of SSSC is finding the maximal effective subset of the current set. As shown in Algorithm 3.3, Emek and Rosén present a method of finding such a subset using a sort, resulting in a time complexity of $O(|S_t| \log |S_t|)$ to process set S_t . Applying SSSC to the unweighted SET COVER problem allows us to improve upon this asymptotic bound. To this end, we first note some observations.

Observation 3.3.1. *For all $1 \leq t \leq m$ and $x \in \mathcal{U}$, $\text{eff}_t(x) \in \{y \in \mathbb{Z} \mid -1 \leq y \leq \text{lev}(\Delta)\}$.*

Observation 3.3.2. *If $x \in S_t$ and $\text{eff}_t(x) \geq \text{lev}(S_t)$, there exists no subset $T \subseteq_{\text{eff}_t} S_t$ where $x \in T$.*

For the weighted SET COVER problem, the level function is given by (the ceiling of) the logarithm of the ratio of two positive rational numbers; clearly, this value is unbounded from below. Per Observation 3.3.1, working with the unweighted version of SET COVER means this is no longer the case. Considering this in combination with Observation 3.2.1 and Observation 3.3.2, we can find the maximal effective subset in linear time by iterating over the frequency distribution of effectiveness values in the set. Algorithm 3.4 demonstrates this, acting as a direct and asymptotically favourable replacement for Algorithm 3.3. Intuitively, this algorithm first initialises with zeroes an array equal in size to the level of the set plus one (this additional one is required to account for the effectiveness of minus one for as-yet-uncovered elements). Then, iterating over the set, the algorithm updates this counter array with the effectiveness of each element, such that the $(i+1)^{\text{th}}$ value in the counter array equates to the number of elements in the set with effectiveness i . Per Observation 3.3.2, any effectiveness which is beyond the bounds of this counter array can safely be omitted. Once the counter array has been populated, the algorithm then iterates over this array, storing the sum of counter values seen so far—this sum corresponds to the size of the would-be subset. The largest effectiveness value which is exceeded by the level of this sum becomes the critical effectiveness value. In a final loop over the set, the algorithm locates the elements whose effectiveness is at most this critical value, adding them to the resulting subset.

Algorithm 3.4: Compute Effective Subset (Count)

```

input :  $S_t \in \mathcal{S}$ , the set at time  $t$ 
output: An effective subset  $T \subseteq_{\text{eff}_t} S_t$  which maximises  $|T|$ 
/* Build the effectiveness frequency distribution */
1 counter  $\leftarrow$  array of size  $\text{lev}(S_t) + 1$ , initialised with zeroes
2 for  $x \in S_t$  do
3   if  $\text{eff}(x) < \text{lev}(S_t)$  then
4     counter[ $\text{eff}(x) + 1$ ] += 1
5   end
6 end
/* Find the critical effectiveness */
7 size  $\leftarrow 0$ , critical_eff  $\leftarrow 0$ 
8 for  $i \leftarrow 0$  to |counter| - 1 do
9   size += counter[ $i$ ]
10  if size > 0 then
11    if  $\text{lev}(\text{size}) > (i - 1)$  then
12      critical_eff  $\leftarrow (i - 1)$ 
13    end
14  end
15 end
/* Populate the result */
16 result  $\leftarrow \emptyset$ 
17 for  $x \in S_t$  do
18   if  $\text{eff}(x) \leq \text{critical\_eff}$  then
19     result  $\leftarrow \text{result} \cup \{x\}$ 
20   end
21 end
22 return result

```

To give an example of the workings of this algorithm, we consider Figure 3.4, which shows an instance of a counter array. Here, the shaded regions of the bars correspond to the counter array values obtained during a first pass of the set, and bars as a whole give the cumulative effectiveness values (i.e., the running sum of counter values) obtained during a second pass. In this instance, the critical effectiveness is 1, as this is the rightmost position where the level of the cumulative counter value exceeds the effectiveness. Our resulting effective subset is therefore of size 3, and a final pass over the set learns the exact elements contained in this subset.

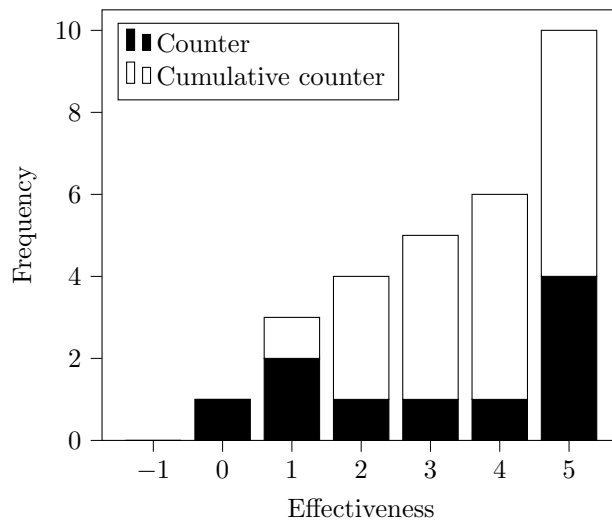


Figure 3.4: An example effectiveness frequency distribution.

Both of these effective subset finding algorithms were implemented. Though this is redundant, they share a common interface and are thus interchangeable, allowing for future empirical comparison.

3.4 Benchmark Data Sets

So as to empirically compare the performance of all considered algorithms, a collection of benchmark data sets is required. These data sets ought not to be synthetic, and should all be large enough in size such that evaluation with exact SET COVER algorithms is impracticable. Indeed, the size of some data sets should exceed the capacity of present-day RAM (8 to 16 gigabytes is typical). Furthermore, the properties displayed by each data set in the collection should vary to account for the range of practical applications that the algorithms may see.

In total, we collated 35 data sets from a range of sources. Included in this sum are the 8 data sets used in the empirical comparison between Greedy and DFG (`chess.dat`, `mushroom.dat`, `pumsb-star.dat`, `pumsb.dat`, `retail.dat`, `accidents.dat`, `kosarak.dat` and `webdocs.dat`) presented by Cormode et al. [16]; this allows us to ensure the solutions produced by our version of DFG approximately match those obtained in the original paper. We briefly outline the key contexts from which the collated data sets originate in Table 3.1 to further note the diversity in potential applications. In some cases, considering the data in set stream form is nonsensical (for example, such is the situation with `chess.dat`, where each set encodes a board permutation in a king and rook versus king and pawn chess endgame), though we do not consider this to be a problem—the exact meaning of the data is arguably irrelevant for the purposes of algorithmic comparison.

Context	Insight given by solving SET COVER in this context
Social network graphs	What subset of network users can together directly reach everyone in the network?
Social network groups	What subset of groups together contain everyone in the network?
Road networks	What collection of junctions are together incident to every junction and dead end on a map?
Web graphs	What subset of webpages together contain a link to every webpage?
Shop baskets	What group of customers together purchased every item on sale?

Table 3.1: Main benchmark data set contexts.

The initial format of data sets varied semantically: some were graphs described as edge lists, others were graphs in adjacency list form, and a few were hypergraphs from the outset. Additionally, there were numerous syntactic variations, including differing or inconsistent delimiters and different line endings. To enable a consistent interface, each data set was modified to fit the same format: a text file with one set to a line (terminated with a line feed), where the elements are space delimited integers. Also, where required, graphs were converted to hypergraphs by aggregating all outgoing edges into a hyperedge for each vertex. These modifications were facilitated by a custom-made C++ script which also performed correctness checks on the data, ensuring that all sets were non-empty and contained no duplicate elements.

3.4.1 Data Set Statistics

To understand the structure of the data sets, a C++ script was written to gather file statistics, including universe size, set count, maximum set size, minimum set size and average set size. These values are given in Table 3.2, sorted by universe size. Additionally, a Python script was created which used `matplotlib` to build histograms showing the distribution of set sizes in each data set.

3.5 Empirical Analysis

To empirically compare DFG and SSSC, we ran our implementations on each of the data sets given in Table 3.2.

All program runs were performed with the system shown in Table 3.3 running under light load. As the size of some data sets exceed the available RAM, the disk-based DFG implementation was required—RAM-based DFG would simply run out of memory. When using this disk-based implementation, we have to carefully consider the value of p : the operating system used permits only 256 open streams (253 if standard in, out and error are excluded from the count), so k must never exceed this value. For this to be guaranteed, we select the value $p = 1.065$, which satisfies $p > \Delta^{1/253}$ for all values of Δ in the benchmark data set collection. We present the results of the all program runs in Table 3.4.

File name	Credit	Size (MB)	$ \mathcal{U} $	$ \mathcal{S} $	$\max S_i $	$\min S_i $	$\text{avg} S_i $
chess	[26]	0.342	75	3196	37	37	37
mushroom	[26]	0.570	119	8124	23	23	23
accidents	[26, 25]	35.5	468	340183	51	18	33
pumsb_star	[26]	11.3	2088	49046	64	49	50
pumsb	[26]	17.6	2113	49046	74	74	74
wikivote_graph	[41, 40]	0.503	2381	6110	893	1	17
retail	[26, 11]	4.2	16470	88162	76	1	10
kosarak	[26]	33.0	41270	990002	2497	1	8
youtube_cmt	[41, 49, 62]	0.824	52675	16386	3001	2	8
gnutella_graph	[41, 56]	0.846	62283	16387	78	1	9
twitter_graph	[41, 43]	16.2	81305	70097	1205	1	25
dblp_cmt	[41, 62]	4.7	260998	13477	7556	6	53
dblp_ungraph	[41, 62]	13.9	317080	317080	343	1	7
amazon_cmt	[41, 62]	15.4	317194	75149	53551	3	30
amazon_ungraph	[41, 62]	12.6	334863	334863	549	1	6
higgs_twitter_graph	[41, 18]	84.5	370341	456626	1259	1	33
berkstan_graph	[41, 42]	51.3	617094	680486	249	1	11
google_graph	[41]	35.2	714545	739454	456	1	7
lj_cmt	[41, 62]	50.4	1147948	664414	178899	2	11
youtube_ungraph	[41, 49, 62]	38.7	1334890	1334890	28754	1	5
wikitc_cmt	[41, 36, 63]	35.1	1791489	17360	418223	1	274
wikitc_graph	[41, 36, 63]	210.8	1791489	1791489	3907	1	16
ca_road_graph	[41, 42]	41.1	1965206	1965206	12	1	3
orkut_cmt	[41, 49, 62]	834.5	2322299	15301901	9120	2	7
wikitalk_graph	[41, 40, 39]	34.7	2369181	147602	100022	1	34
orkut_ungraph	[41, 49, 62]	1770	3072441	3072441	33313	1	76
lj_ungraph	[41, 62]	501.6	3997962	3997962	14815	1	17
lj1_graph	[41, 42]	504.9	4489240	4308452	20293	1	16
webdocs	[26, 15]	1482	5267656	1629082	71472	1	177
friendster_cmt	[41, 62]	210.9	7944949	1620991	9299	2	14
orkut_u2g_graph	[57, 49]	2198	8730857	2783196	40425	1	118
twitter	[38]	12035	35689148	40103281	2997469	1	37
web_pld_graph	[57, 46]	5447	36165039	27370614	3898561	1	23
friendster_ungraph	[41, 62]	32365	65608366	65608366	5214	1	55
flsd_graph	[57, 46]	15724	66365866	59681219	982117	1	30

Table 3.2: Benchmark data set statistics.

Operating System	macOS Mojave 10.14.6
Processor Name	Intel Core i5
Processor Speed	2.4GHz
Number of Cores	4
Level 2 Cache (per Core)	256KB
Level 3 Cache	6MB
RAM	16GB
Disk	512GB (SSD)

Table 3.3: Specification of system used for experiments.

File name	Cover size		Time taken (s)		Peak RAM (MB)		Disk (MB)
	DFG	SSSC	DFG	SSSC	DFG	SSSC	DFG
chess	8	13	0.03	0.01	1.08	0.85	2.67
mushroom	22	27	0.09	0.02	1.11	0.88	6.88
accidents	183	213	3.79	0.99	1.26	0.94	347.30
pumsb_star	743	904	0.77	0.22	1.31	1.04	89.18
pumsb	752	904	0.89	0.28	1.36	1.06	105.09
wikivote_graph	174	236	0.08	0.01	1.81	1.06	2.17
retail	5130	5908	0.33	0.15	2.32	2.09	26.16
kosarak	17746	18618	1.90	1.19	4.42	4.06	172.83
youtube_cmt	9402	9497	0.11	0.04	5.12	4.78	4.09
gnutella_graph	13448	13839	0.14	0.05	4.99	5.21	7.02
twitter_graph	1003	1529	0.55	0.30	6.09	5.43	36.01
dblp_cmt	11283	11409	0.32	0.15	17.60	16.84	22.76
dblp_ungraph	50372	59541	1.01	0.61	18.45	20.15	64.78
amazon_cmt	3577	3851	0.61	0.36	20.98	19.42	43.80
amazon_ungraph	62885	77893	1.03	0.78	18.99	21.49	60.93
higgs_twitter_graph	80748	90555	3.85	2.07	20.44	22.97	355.00
berkstan_graph	131112	140399	2.90	1.29	34.82	39.44	207.13
google_graph	147894	161983	2.67	1.76	37.97	43.70	150.74
lj_cmt	147580	153321	3.10	2.47	71.72	75.68	192.21
youtube_ungraph	231707	238619	2.51	1.76	66.42	74.65	184.92
wikit_cmt	14085	14496	1.49	0.80	125.45	128.19	116.27
wikit_graph	264547	295132	10.08	5.73	113.07	123.30	703.07
ca_road_graph	792474	833385	3.71	2.21	117.63	147.60	232.95
orkut_cmt	149239	158439	39.84	33.09	131.02	135.95	2809.05
wikitalk_graph	32806	32890	2.97	1.44	136.11	135.39	155.75
orkut_ungraph	105679	229409	55.19	39.19	156.89	165.81	5096.13
lj_ungraph	794624	941801	27.41	18.03	238.14	272.60	1935.83
lj1_graph	956608	1096844	28.63	17.60	254.56	294.46	1938.41
webdocs	406375	413819	38.23	20.25	282.94	295.11	5396.66
friendster_cmt	769305	781962	18.71	14.13	471.70	501.98	770.52
orkut_u2g_graph	916457	994050	104.62	48.28	498.28	532.38	9817.80
twitter	3845642	4284164	403.17	372.90	1718.85	1902.38	34253.70
web_pld_graph	4117387	4328159	192.73	191.80	1831.62	1905.78	13518.30
friendster_ungraph	10763395	12434527	1609.81	1063.10	3275.10	3639.46	95484.10
flsd_graph	9070300	9593175	554.00	509.59	3281.15	3581.56	39198.50

Table 3.4: A direct performance comparison between disk-based DFG ($p = 1.065$) and SSSC.

Some clear patterns emerge. To start, the covers produced by DFG are better in all cases; this is to be expected when considering the theoretical guarantees of the greedy heuristic. Importantly, however, the sizes of the covers produced by SSSC are, in most cases, very close to those given by DFG. Specifically, the covers produced by SSSC are on average only a factor of 1.16 larger than corresponding covers from DFG. This is very promising, especially when considering that these covers were obtained by SSSC without use of any external memory. A second clear difference is the time taken: considering all cases, SSSC produces a solution on average 2.12 times faster—and never slower—than DFG. Finally, both SSSC and DFG consume a similar amount of RAM on average. This is unsurprising as both algorithms store a copy of every element in the input universe at some point during execution. However, RAM usage increases at a slightly higher rate with SSSC as problem sizes grow—this is due to the additional post-processing step of extracting a list of set identifiers from the cover certificate. Given these results, it is clear that SSSC is a highly capable algorithm for the SET COVER problem in practice.

3.6 Approximation Factor

After establishing the capabilities of SSSC in practice, we now turn our attention to the theoretical guarantees. Emek and Rosén show that SSSC produces a solution within a factor $9\sqrt{n}$ of $|\text{OPT}|$ for

weighted SET COVER. Simplifying the context to the unweighted SET COVER problem has allowed for a tightening of this bound, which we now state in terms of Δ . By modifying the analysis presented by Emek and Rosén, we now show that SSSC produces a solution within a factor $4\sqrt{2}\sqrt{\Delta} - 1$ of $|\text{OPT}|$.

As SSSC guarantees that effectiveness is non-decreasing throughout execution, Observation 3.6.1 follows from Observation 3.2.1.

Observation 3.6.1. *If $T \subseteq_{\text{eff}_t} S_t$, it holds that $\text{eff}_{t+1}(x) \geq \text{lev}(T)$ for every $x \in T$.*

The following lemma can now be established.

Lemma 3.6.1. *Consider some $r \in \mathbb{R}$. SSSC guarantees that*

$$|\{x \in S_t \mid \text{eff}_{t+1}(x) \leq r\}| < 2^{r+1}.$$

Proof. Assume by contradiction that this is not the case, thus there exists some subset $R \subseteq S_t$ where $|R| \geq 2^{r+1}$, such that $\text{eff}_{t+1}(x) \leq r$ for every $x \in R$. Effectiveness is non-decreasing, therefore $\text{eff}_t(x) \leq r$ for every $x \in R$, ensuring that R is effective at time t . However, this directly contradicts Observation 3.6.1: $\text{eff}_{t+1}(x)$ should be at least $\lceil r + 1 \rceil$ for every $x \in R$. $\square$

For some $x \in \mathcal{U}$, let $\text{eff}_\infty(x)$ denote the value of $\text{eff}(x)$ upon termination of the input stream.

Definition 3.6.1 ($I(r)$). For some $r \in \mathbb{Z}$, let

$$I(r) = \{x \in \mathcal{U} \mid \text{eff}_\infty(x) = r\}.$$

Definition 3.6.2 ($S(r)$). For some $r \in \mathbb{Z}$, let

$$S(r) = \{S \in \mathcal{S} \mid \exists x \in I(r) \text{ s.t. } \text{eid}(x) = \text{id}(S)\}.$$

Definition 3.6.1 and Definition 3.6.2 are extended to the real interval $[a, b]$ as follows:

$$\begin{aligned} I([a, b]) &= \{x \in \mathcal{U} \mid a \leq \text{eff}_\infty(x) \leq b\}, \\ S([a, b]) &= \{S \in \mathcal{S} \mid \exists x \in I([a, b]) \text{ s.t. } \text{eid}(x) = \text{id}(S)\}. \end{aligned}$$

The real intervals $(-\infty, r]$ and (r, ∞) are denoted by $\leq r$ and $> r$ respectively.

Lemma 3.6.2. *Consider some $r \in \mathbb{R}$. SSSC guarantees that*

$$|I(\leq r)| < 2^{r+1} \cdot |\text{OPT}|.$$

Proof. Since the effectiveness is non-decreasing, Lemma 3.6.1 ensures that for every set $S \in \mathcal{S}$, it holds that

$$|\{x \in S \mid \text{eff}_\infty(x) \leq r\}| < 2^{r+1}.$$

As OPT is a cover of $\mathcal{U}$, every element contained in $I(\leq r)$ also appears at least once in the sets comprising OPT . It follows that

$$|I(\leq r)| \leq \sum_{S \in \text{OPT}} |\{x \in S \mid \text{eff}_\infty(x) \leq r\}| < \sum_{S \in \text{OPT}} 2^{r+1} = 2^{r+1} \cdot |\text{OPT}|.$$

$\square$

Lemma 3.6.3. *Consider some $r \in \mathbb{Z}$. The edge collection $S(r)$ satisfies*

$$|S(r)| < \frac{n}{2^{r-1}}.$$

Proof. If $S_t \in S(r)$, then there exists some subset $R_{S_t} \subseteq_{\text{eff}_t} S_t$ with $\text{lev}(R_{S_t}) = r$. This subset satisfies $2^{r-1} < |R_{S_t}|$ by definition of level, and for every element $x \in R_{S_t}$ we have (1) $\text{eff}_t(x) < r$; and (2) $\text{eff}_{t+1}(x) = r$. As $\text{eff}(x)$ is non-decreasing and $\text{eid}(x)$ is updated only when $\text{eff}(x)$ increases, it follows that if $S_t, S_{t'} \in S(r)$ such that $S_t \neq S_{t'}$, then the subsets R_{S_t} and $R_{S_{t'}}$ are disjoint. Subsequently,

$$|S(r)| = \sum_{S_t \in S(r)} 1 < \frac{\sum_{S_t \in S(r)} |R_{S_t}|}{2^{r-1}} \leq \frac{n}{2^{r-1}}.$$

$\square$

Lemma 3.6.4. *Consider some $r \in \mathbb{R}$. The edge collection $S(>r)$ satisfies*

$$|S(>r)| < \left(\frac{\Delta}{2^{r-2}} - 1 \right) \cdot |\text{OPT}|.$$

Proof. By Observation 3.3.1, we know that $|I(>\lceil \log_2 \Delta \rceil)| = 0$. Summing the bound given in Lemma 3.6.3 over the integer interval $[\lceil r \rceil + 1, \lceil \log_2 \Delta \rceil]$ results in the converging series

$$\sum_{r'=\lceil r \rceil+1}^{\lceil \log_2 \Delta \rceil} \frac{n}{2^{r'-1}} = n \left(\frac{1}{2^{\lceil r \rceil-1}} - \frac{1}{2^{\lceil \log_2 \Delta \rceil-1}} \right) < n \left(\frac{1}{2^{r-2}} - \frac{1}{\Delta} \right) \leq \left(\frac{\Delta}{2^{r-2}} - 1 \right) \cdot |\text{OPT}|.$$

□

Theorem 3.6.1. *The cardinality of the solution produced by SSSC is within a factor $4\sqrt{2}\sqrt{\Delta} - 1$ of $|\text{OPT}|$.*

Proof. Consider some $r \in \mathbb{R}$. By combining the bounds of the solution subsets addressed in Lemma 3.6.2 and Lemma 3.6.4, and noting that $|S(r)| \leq |I(r)|$, the cover certificate χ returned by SSSC satisfies

$$|\text{Im}(\chi)| < \left(2^{r+1} + \frac{\Delta}{2^{r-2}} - 1 \right) \cdot |\text{OPT}|.$$

This approximation factor is minimised, i.e.,

$$\frac{\partial}{\partial r} \left(2^{r+1} + \frac{\Delta}{2^{r-2}} - 1 \right) = \left(2^{r+1} - \frac{\Delta}{2^{r-2}} \right) \ln 2 = 0,$$

when $r = \log_2 \sqrt{2\Delta}$. Using this value for r , it follows that

$$|\text{Im}(\chi)| < \left(4\sqrt{2}\sqrt{\Delta} - 1 \right) \cdot |\text{OPT}|,$$

which completes the proof. □

3.6.1 Accounting for Disproportionality

It is sometimes the case that the size of the largest set is not representative of the set size distribution. To give an example, the largest set in the file `wikitc.cmtty.dat` has size $\Delta = 418223$, which is significantly larger than 9467, the size of the second largest second largest set.

We know that all instances of SET COVER satisfy $|\text{OPT}| \geq n/\Delta$, however, as the value of Δ disproportionately increases, the tightness of this bound quickly deteriorates. If we sort in descending size order a given problem instance $\mathcal{S}$, it follows that $\Delta = |S_1| \geq |S_2| \geq \dots \geq |S_m|$, where $S_i \in \mathcal{S}$ for each $1 \leq i \leq m$. If c is the smallest integer which satisfies $\sum_{i=1}^c |S_i| \geq n$, then we know that $|\text{OPT}| \geq c$. We now introduce an instance-specific value

$$\Delta' = \frac{n}{c} \geq \frac{n}{|\text{OPT}|},$$

which has the potential to be much smaller than Δ (in the case of `wikitc.cmtty.dat`, $\Delta' \approx 1760$). Incorporating this into our approximation factor analysis requires a minor modification to Lemma 3.6.4, and, in turn, Theorem 3.6.1.

Lemma 3.6.5. *Consider some $r \in \mathbb{R}$. The edge collection $S(>r)$ satisfies*

$$|S(>r)| < \left(\frac{\Delta'}{2^{r-2}} \right) \cdot |\text{OPT}|.$$

Proof. Summing the bound given in Lemma 3.6.3 over the integer interval $[\lceil r \rceil + 1, \infty)$ results in the converging series

$$\sum_{r'=\lceil r \rceil+1}^{\infty} \frac{n}{2^{r'-1}} = \frac{n}{2^{\lceil r \rceil-1}} < \frac{n}{2^{r-2}} \leq \left(\frac{\Delta'}{2^{r-2}} \right) \cdot |\text{OPT}|$$

□

Theorem 3.6.2. *The cardinality of the solution produced by SSSC is within a factor $4\sqrt{2}\sqrt{\Delta'}$ of $|\text{OPT}|$.*

Proof. Consider some $r \in \mathbb{R}$. By combining the bounds of the solution subsets addressed in Lemma 3.6.2 and Lemma 3.6.5, and noting that $|S(r)| \leq |I(r)|$, the cover certificate χ returned by SSSC satisfies

$$|\text{Im}(\chi)| < \left(2^{r+1} + \frac{\Delta'}{2^{r-2}}\right) \cdot |\text{OPT}|,$$

which, when minimised by $r = \log_2 \sqrt{2\Delta'}$, gives

$$|\text{Im}(\chi)| < \left(4\sqrt{2}\sqrt{\Delta'}\right) \cdot |\text{OPT}|.$$

□

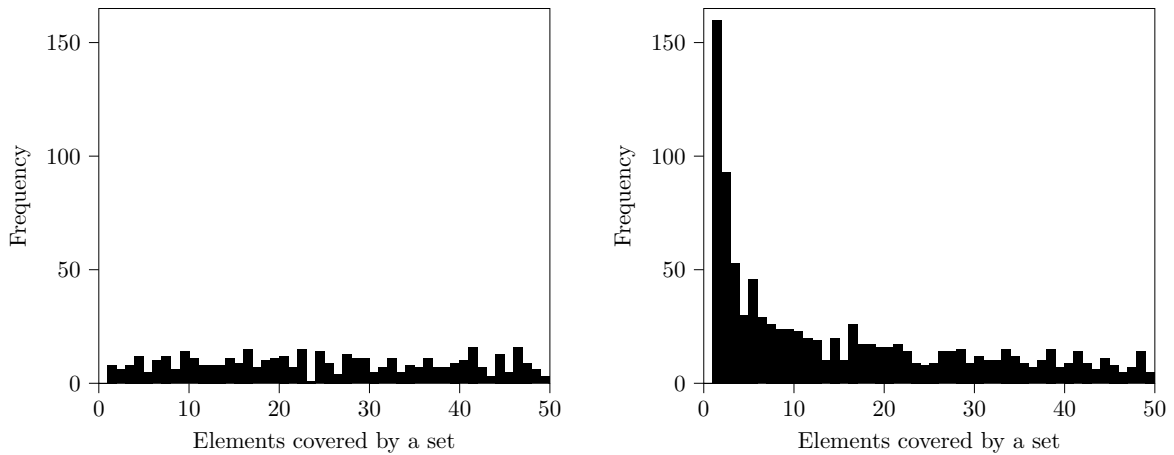
Chapter 4

Multiple Passes

In contrast to DFG, which allows for an improved approximation factor at the expense of computation time by introducing a parameter p , SSSC as originally given is not parameterised, with no clear way of reducing the resulting cover size short of implementing an alternative algorithm. In this chapter, we motivate, describe and analyse (both empirically and theoretically) a new algorithm which admits improved solution sizes by generalising SSSC to multiple passes.

4.1 Generalising SSSC to Multiple Passes

To learn how best to improve SSSC, we first demonstrate why the solution sizes are larger than those produced by DFG. For this purpose, we define the *contribution* of a set S to be the number of elements for which S is the designated coverer; clearly, unless necessary, having many sets with a low contribution is undesirable. Extracting the distribution of contributions from a DFG cover is simple: we note how many uncovered elements each set contained at the point when it was added to the solution. For SSSC, the contribution distribution of a solution can be learnt by considering the inverse mapping of the cover certificate, i.e., evaluating $|\{x \in \mathcal{U} \mid \text{eid}(x) = \text{id}(S)\}|$ for each set identifier $\text{id}(S)$ in the solution. Histograms describing the contribution distributions for both DFG and SSSC are given for the file `twitter_graph.dat` (see Appendix A for the set size distribution of this file) in Figure 4.1, where only the lower end of the distribution is shown. This particular file was chosen as the cover produced by DFG in this instance has a fairly uniform distribution, which eases comparison with the corresponding SSSC distribution. It is clear that many sets in the SSSC solution are not as effective as they were at an earlier point in execution; the proportion of sets included in the solution only to cover a few elements is much higher for SSSC than DFG. With this in mind, if we could somehow better-cover the elements which reside at the lower end of the cover contribution distribution, then we would be able to reduce the total set count in the solution.



(a) DFG ($p = 1 + 10^{-6}$) cover contribution distribution

(b) SSSC cover contribution distribution

Figure 4.1: A comparison between the contribution distributions given by SSSC and DFG for the file `twitter_graph.dat`.

To this end, we introduce Multi-pass Semi-Streaming Set Cover (MSSSC). The core concept of MSSSC is simple: on the first pass, normal SSSC is applied to the set stream. Then, based on effectiveness, we restrict the universe to some subset of the original (i.e., all elements whose effectiveness is at most some threshold r), and only those sets covering the elements not in this restricted subuniverse are added to the solution. SSSC is then applied to the restricted universe on the second pass, after which the universe is again restricted, and so this recursive approach continues for a total of p passes. This idea is illustrated in Figure 4.2, where $\mathcal{U}_j$ is the universe at the beginning of the j^{th} pass, i.e., $\mathcal{U}$ as restricted by $j - 1$ previous passes. In this diagram, each shaded region depicts the restricted universe for the next pass, and the covers for the clear regions together comprise the solution returned by MSSSC.

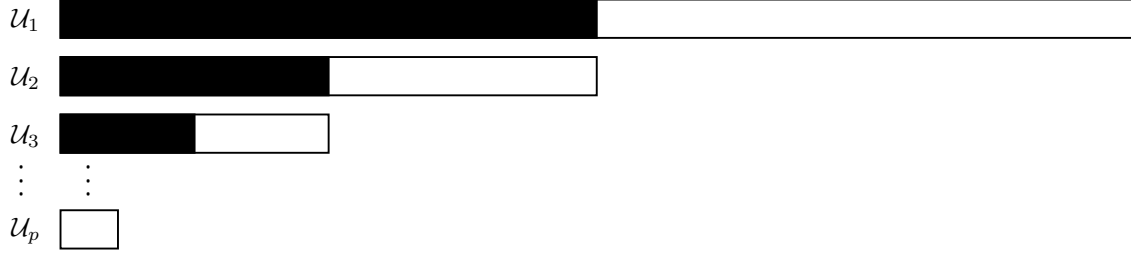


Figure 4.2: Universe decomposition for p passes of MSSSC.

The question now is how to choose a suitable universe restriction threshold r_j for the j^{th} pass, and, to answer this, we must consider the approximation guarantees for p -pass MSSSC.

Theorem 4.1.1. *The cardinality of the solution produced by p -pass MSSSC is within a factor $(p + 1)8^{p/(p+1)}\Delta^{1/(p+1)} - p$ of $|\text{OPT}|$.*

Proof. We prove this by induction. Let $\alpha_p = (p + 1)8^{p/(p+1)}\Delta^{1/(p+1)} - p$.

Inductive hypothesis. For some $p \geq 1$, p -pass MSSSC produces a solution within a factor α_p of $|\text{OPT}|$.

Base case. Let us first consider the base case $p = 1$. The approximation factor is given by

$$\alpha_1 = (1 + 1)8^{1/(1+1)}\Delta^{1/(1+1)} - 1 = 4\sqrt{2}\sqrt{\Delta} - 1,$$

which matches the approximation factor given in Theorem 3.6.1.

Inductive step. Assume the inductive hypothesis is true for $p = k$, i.e.,

$$\alpha_k = (k + 1)8^{k/(k+1)}\Delta^{1/(k+1)} - k.$$

We now show that the induction hypothesis holds for $p = k + 1$.

Consider some threshold $r \in \mathbb{R}$. In the first of our $k + 1$ passes, we keep only those sets in $S(> r)$ in our final solution $\text{Im}(\chi)$, leaving the subproblem of finding a cover for the remaining subuniverse $I(\leq r)$. As we have k passes remaining, we now apply k -pass MSSSC to this subproblem, which we know under our assumption produces a solution of size less than

$$\left((k + 1)8^{k/(k+1)}\Delta_2^{1/(k+1)} - k \right) \cdot |\text{OPT}_2|,$$

where Δ_2 corresponds to the maximum number of elements in $I(\leq r)$ contained in a single set and OPT_2 is the optimal covering of $I(\leq r)$ which satisfies $|\text{OPT}_2| \leq |\text{OPT}|$. As effectiveness is non-decreasing, by Lemma 3.6.1 we know that every set contains fewer than 2^{r+1} elements of $I(\leq r)$, therefore $\Delta_2 < 2^{r+1}$. Considering this alongside Lemma 3.6.4, it follows that

$$|\text{Im}(\chi)| < \left((k + 1)8^{k/(k+1)} (2^{r+1})^{1/(k+1)} - k + \frac{\Delta}{2^{r-2}} - 1 \right) \cdot |\text{OPT}|.$$

This approximation factor is minimised, i.e.,

$$\begin{aligned} \frac{\partial}{\partial r} \left((k+1)8^{k/(k+1)} (2^{r+1})^{1/(k+1)} - k + \frac{\Delta}{2^{r-2}} - 1 \right) \\ = \left(8^{k/(k+1)} (2^{r+1})^{1/(k+1)} - \frac{\Delta}{2^{r-2}} \right) \ln 2 \\ = 0, \end{aligned}$$

when the threshold r is set to

$$r = \frac{(k+1) \log_2 \Delta + 1 - k}{k+2}.$$

With this threshold,

$$\begin{aligned} |\text{Im}(\chi)| &< \left((k+1)8^{k/(k+1)} \left(2^{((k+1) \log_2 \Delta + 1 - k)/(k+2) + 1} \right)^{1/(k+1)} - k \right. \\ &\quad \left. + \frac{\Delta}{2^{((k+1) \log_2 \Delta + 1 - k)/(k+2) - 2}} - 1 \right) \cdot |\text{OPT}|, \end{aligned}$$

which eventually simplifies to

$$|\text{Im}(\chi)| < \left((k+2)8^{(k+1)/(k+2)} \Delta^{1/(k+2)} - k - 1 \right) \cdot |\text{OPT}|.$$

This approximation factor exactly matches α_{k+1} , thus showing that α_{k+1} indeed holds.

Conclusion. As both the base case and inductive step hold, by the principle of mathematical induction p -pass MSSSC produces a solution of size within a factor $(p+1)8^{p/(p+1)} \Delta^{1/(p+1)} - p$ of $|\text{OPT}|$. $\square$

Corollary 4.1.1.1. *The cardinality of the solution produced by p -pass MSSSC is within a factor $8(p+1)\Delta^{1/(p+1)}$ of $|\text{OPT}|$.*

Proof. This simplification (at the expense of tightness) can be established by observing that

$$\lim_{p \rightarrow \infty} 8^{p/(p+1)} = 8,$$

which admits

$$\begin{aligned} |\text{Im}(\chi)| &< \left((p+1)8^{p/(p+1)} \Delta^{1/(p+1)} - p \right) \cdot |\text{OPT}| \\ &\leq \left(8(p+1)\Delta^{1/(p+1)} - p \right) \cdot |\text{OPT}| \\ &< \left(8(p+1)\Delta^{1/(p+1)} \right) \cdot |\text{OPT}|. \end{aligned}$$

$\square$

Given that we now have an approximation factor for p -pass MSSSC, we can derive a formula to calculate a suitable threshold r_j for the j^{th} pass. To this end, let Δ_j be the maximum number of elements in $\mathcal{U}_j$ contained in a single set, where $\Delta_1 = \Delta$. More formally, $\Delta_j = \max_{S \in \mathcal{S}} |S \cap \mathcal{U}_j|$.

Lemma 4.1.1. *To achieve the bound given in Theorem 4.1.1, the threshold r_j for the j^{th} pass of p -pass MSSSC is given by*

$$r_j = \frac{(p-j+1) \log_2 \Delta_1 + 3j - p - 1}{p+1}.$$

Proof. We prove this by induction. Let $\varphi_j = \frac{(p-j+1) \log_2 \Delta_1 + 3j - p - 1}{p+1}$.

Inductive hypothesis. For some $1 \leq j \leq p$, the approximation factor given by Theorem 4.1.1 for the j^{th} pass of p -pass MSSSC is minimised with the threshold $r_j = \varphi_j$.

Base case. Let us first consider the base case $j = 1$. By Theorem 4.1.1, we know the approximation factor of $(p-1)$ -pass MSSSC, thus the approximation factor of p -pass MSSSC is of the form

$$|\text{Im}(\chi)| < \left(p 8^{(p-1)/p} \Delta_2^{1/p} + \frac{\Delta_1}{2^{r_1-2}} - p \right) \cdot |\text{OPT}|$$

for some $r_1 \in \mathbb{R}$. This value r_1 gives the threshold for which elements with an effectiveness greater than r_1 after the first pass are added to the solution (and hence not present in $\mathcal{U}_2$). As effectiveness is non-decreasing, by Lemma 3.6.1 we know that $\Delta_2 < 2^{r_1+1}$, so this expression is minimised, i.e.,

$$\begin{aligned} \frac{\partial}{\partial r_1} \left(p 8^{(p-1)/p} (2^{r_1+1})^{1/p} + \frac{\Delta_1}{2^{r_1-2}} - p \right) \\ = \left(8^{(p-1)/p} (2^{r_1+1})^{1/p} - \frac{\Delta_1}{2^{r_1-2}} \right) \ln 2 \\ = 0, \end{aligned}$$

when $r_1 = \frac{p \log_2 \Delta_1 + 2 - p}{p+1} = \varphi_1$.

Inductive step. Assume the inductive hypothesis is true for $j = k$.

After $(k-1)$ previous passes, the k^{th} pass processes the restricted universe $\mathcal{U}_k$ and produces a corresponding cover certificate χ_k . By Theorem 4.1.1, the approximation factor for the k^{th} pass is given by

$$|\text{Im}(\chi_k)| < \left((p-k+1) 8^{(p-k)/(p-k+1)} \Delta_{k+1}^{1/(p-k+1)} - (p-k+1) + \frac{\Delta_k}{2^{r_k-2}} - 1 \right) \cdot |\text{OPT}_k|$$

for some $r_k \in \mathbb{R}$, where OPT_k is the optimal covering of $\mathcal{U}_k$. Again by Lemma 3.6.1, we know that $\Delta_{k+1} < 2^{r_k+1}$, and, given our assumption, we know that the approximation factor above is minimised when $r_k = \varphi_k$, so $\Delta_{k+1} < 2^{\varphi_k+1}$.

We now show that the induction hypothesis holds for $j = k+1$. The approximation factor for this pass is

$$\begin{aligned} |\text{Im}(\chi_{k+1})| &< \left((p-k) 8^{(p-k-1)/(p-k)} \Delta_{k+2}^{1/(p-k)} - (p-k) + \frac{\Delta_{k+1}}{2^{r_{k+1}-2}} - 1 \right) \cdot |\text{OPT}_{k+1}| \\ &< \left((p-k) 8^{(p-k-1)/(p-k)} (2^{r_{k+1}+1})^{1/(p-k)} + \frac{2^{\varphi_k+1}}{2^{r_{k+1}-2}} + k - p - 1 \right) \cdot |\text{OPT}_{k+1}|. \end{aligned}$$

This approximation factor is minimised, i.e.,

$$\begin{aligned} \frac{\partial}{\partial r_{k+1}} \left((p-k) 8^{(p-k-1)/(p-k)} (2^{r_{k+1}+1})^{1/(p-k)} + 2^{\varphi_k-r_{k+1}+3} + k - p - 1 \right) \\ = \left(8^{(p-k-1)/(p-k)} (2^{r_{k+1}+1})^{1/(p-k)} - 2^{\varphi_k-r_{k+1}+3} \right) \ln 2 \\ = 0, \end{aligned}$$

when $r_{k+1} = \frac{\varphi_k(p-k)+2}{p-k+1} = \varphi_{k+1}$.

Conclusion. As both the base case and inductive step hold, by the principle of mathematical induction the bound given in Theorem 4.1.1 is achieved with the threshold $r_j = \varphi_j$ for each of the j passes of p -pass MSSSC. $\square$

Having established a suitable threshold, we now present Algorithm 4.1, which gives a more formal non-recursive definition of MSSSC. Note now that sets added to the final solution are never subsequently removed; as a result, if constructed alongside the cover certificate during execution, a list of identifiers $\mathcal{I}$ can be returned directly if desired.

Algorithm 4.1: Multi-pass Semi Streaming Set Cover

```

input : A stream of sets  $\mathcal{S}$ 
output: A cover certificate of  $\mathcal{S}$  alongside final effectiveness values
1 foreach  $x \in \mathcal{U}$  do  $\text{eid}(x) \leftarrow \text{NULL}$ ;  $\text{eff}(x) \leftarrow -1$ 
2 for  $j \leftarrow 1$  to  $p$  do
3    $e \leftarrow$  final effectiveness values returned by a pass of SSSC with universe restricted to  $\mathcal{U}$ 
4    $\chi \leftarrow$  final cover certificate returned by a pass of SSSC with universe restricted to  $\mathcal{U}$ 
5    $\mathcal{U}' \leftarrow \emptyset$ 
6    $r \leftarrow ((p - j + 1) \log_2 \Delta_1 + 3j - p - 1) / (p + 1)$ 
7   for  $x \in \mathcal{U}$  do
8     if  $j < p$  and  $e(x) \leq r$  then
9        $\mathcal{U}' \leftarrow \mathcal{U}' \cup \{x\}$ 
10    else
11       $\text{eff}(x) \leftarrow e(x)$ 
12       $\text{eid}(x) \leftarrow \chi(x)$ 
13    end
14  end
15   $\mathcal{U} \leftarrow \mathcal{U}'$ 
16 end
17 return  $\text{eid}(\cdot)$  and  $\text{eff}(\cdot)$ 

```

Chakrabarti and Wirth [12] prove that a semi-streaming algorithm cannot achieve an approximation factor better than $0.99n^{1/(p+1)}/(p+1)^2$, thus MSSSC is tight up to a factor of $8(p+1)^3$. We now wish to learn whether MSSSC performs well in practice, so to do this we compare it with an alternative algorithm which executes under the same model. We start by introducing this algorithm.

4.2 Progressive Greedy

Introduced by Chakrabarti and Wirth [12], Progressive Greedy is also multi-pass semi-streaming algorithm for the SET COVER problem. Conceptually, Progressive Greedy is very simple, taking a similar approach to the greedy algorithm described in Section 2.4. As in the paper which presents Progressive Greedy, we first describe a naïve version of the algorithm (Naïve Progressive Greedy), after which we introduce the complicating modification which allows us to save a pass over the input stream.

During the first pass, Naïve Progressive Greedy selects a suitable threshold value τ_1 , adding sets to the solution if they contain τ_1 or more as-yet-uncovered elements. The second pass repeats this logic with a smaller threshold τ_2 , and the algorithm continues in this way for a total of p passes over the stream, with $\tau_p = 1$. Using the threshold

$$\tau_j = n^{1 - \frac{j}{p}}$$

for the j^{th} pass, as implemented in Algorithm 4.2, Chakrabarti and Wirth demonstrate that this gives a $pn^{1/p}$ -approximation to the SET COVER problem.

Note that the final pass of this naïve algorithm simply assigns sets containing a non-zero count of uncovered elements to the solution. For this reason, the last two pass can be *folded* into a single pass: the would-be penultimate pass continues with the threshold τ_{p-1} , and—in parallel—also builds a cover certificate from scratch with a threshold of 1. This secondary cover certificate is then appropriately combined with the main cover certificate to produce the solution. This is the technique employed by Progressive Greedy (shown in Algorithm 4.3) to improve upon the naïve algorithm; for every input stream, Chakrabarti and Wirth show that p -pass Progressive Greedy produces a solution identical to that given by $(p+1)$ -pass Naïve Progressive Greedy. As such, the approximation factor of Progressive Greedy is $(p+1)n^{p+1}$.

As sets are never removed from the solution during execution, Progressive Greedy is able to return both a cover certificate and a list of identifiers; though the latter can be extracted from the former, returning both eliminates the need for any post-processing. We note that, while Naïve Progressive Greedy is able to give a solution without generating a cover certificate, this is not the case for Progressive Greedy—the secondary cover certificate created during the folded pass is required. In terms of asymptotic complexities, Progressive Greedy processes each set in expected linear time, and uses $\Theta(n \log m)$ bits of space. We also note that, unlike SSSC and MSSSC, Progressive Greedy requires prior knowledge of the stream; the value of n is needed to calculate a suitable threshold for the first pass.

Algorithm 4.2: Naïve p -pass Progressive Greedy

```

input : A stream of sets  $\mathcal{S}$ 
output: A cover certificate of  $\mathcal{S}$  and a collection of identifiers  $\mathcal{I}$  which together cover  $\mathcal{U}$ 
1 function GreedyPass(set stream  $\mathcal{S}$ , threshold  $\tau$ , solution set  $\mathcal{I}$ , cover certificate  $\chi$ ):
2   for  $t \leftarrow 1$  to  $m$  do
3     Read the set  $S_t$  from  $\mathcal{S}$ 
4      $C \leftarrow \{x \in \mathcal{U} \mid \chi(x) \neq \text{NULL}\}$ 
5     if  $|S_t \setminus C| \geq \tau$  then
6        $\mathcal{I} \leftarrow \mathcal{I} \cup \{\text{id}(S_t)\}$ 
7       for  $x \in S_t$  do
8          $\chi(x) \leftarrow \text{id}(S_t)$ 
9       end
10    end
11  end
12 end
13  $\mathcal{I} \leftarrow \emptyset$ 
14 foreach  $x \in \mathcal{U}$  do  $\chi(x) \leftarrow \text{NULL}$ 
15 for  $j \leftarrow 1$  to  $p$  do
16   GreedyPass( $\mathcal{S}$ ,  $n^{1-j/p}$ ,  $\mathcal{I}$ ,  $\chi$ )
17 end
18 return  $\chi(\cdot)$  and  $\mathcal{I}$ 
    
```

Algorithm 4.3: p -pass Progressive Greedy

```

input : A stream of sets  $\mathcal{S}$ 
output: A cover certificate of  $\mathcal{S}$  and a collection of identifiers  $\mathcal{I}$  which together cover  $\mathcal{U}$ 
1  $\mathcal{I}, \mathcal{I}' \leftarrow \emptyset$ 
2 foreach  $x \in \mathcal{U}$  do  $\chi(x), \chi'(x) \leftarrow \text{NULL}$ 
3 for  $j \leftarrow 1$  to  $p-1$  do
4   GreedyPass( $\mathcal{S}$ ,  $n^{1-j/(p+1)}$ ,  $\mathcal{I}$ ,  $\chi$ )
5 end
6 /* Fold the last two passes into one */
7 do in parallel
8   GreedyPass( $\mathcal{S}$ ,  $n^{1-p/(p+1)}$ ,  $\mathcal{I}$ ,  $\chi$ )
9   GreedyPass( $\mathcal{S}$ , 1,  $\mathcal{I}'$ ,  $\chi'$ )
10 end
11 /* Post processing step to merge secondary cover certificate  $\chi'$  into  $\chi$  */
12 for  $x \in \mathcal{U}$  do
13   if  $\chi(x) = \text{NULL}$  then
14      $\chi(x) \leftarrow \chi'(x)$ 
15      $\mathcal{I} \leftarrow \mathcal{I} \cup \{\chi'(x)\}$ 
16   end
17 end
18 return  $\chi(\cdot)$  and  $\mathcal{I}$ 
    
```

4.2.1 Approximation Factor

Progressive Greedy was originally presented with an approximation factor in terms of the universe size n . We restate these guarantees in terms of the maximum set size Δ .

We first consider Naïve Progressive Greedy as given in Algorithm 4.3, which has an approximation factor of $pn^{1/p}$ when using the threshold $\tau_j = n^{1-j/p}$ for the j^{th} pass. With a very minor modification to both the algorithm and the proof given by Chakrabarti and Wirth, a similar approximation guarantee can be given in terms of the cardinality of the largest set: we now restate the original proofs to show that using the threshold $\tau_j = \Delta^{1-j/p}$ for the j^{th} pass results in an approximation factor of $p\Delta^{1/p}$. With this result, an approximation factor of $(p+1)\Delta^{1/(p+1)}$ for Progressive Greedy follows.

We reiterate that the modification to the analysis originally given by Chakrabarti and Wirth is very minor—the structure of the proof remains unchanged, and is thus not a contribution of our own.

Definition 4.2.1 ((τ, ρ) -bound). A pass of the set stream $\mathcal{S}$ with threshold τ is said to be (τ, ρ) -bounded if for all $S \in \mathcal{S}$, $|S \setminus C_0| \leq \rho\tau$, where C_0 is the set of elements covered prior to traversal of the stream.

Lemma 4.2.1. A (τ, ρ) -bounded pass adds at most $\rho|\text{OPT}|$ sets to $\mathcal{I}$.

Proof. Let $D = \mathcal{U} \setminus C_0$, the set of elements left uncovered at the start of this pass. By Definition 4.2.1, we know that no set in $\mathcal{S}$ contains more than $\rho\tau$ of the elements in D , thus OPT covers D with at least $|D|/\rho\tau$ sets. In this pass, the logic of Naïve Progressive Greedy ensures that each set added to $\mathcal{I}$ includes at least τ elements. Therefore, the pass adds at most $|D|/\tau \leq \rho|\text{OPT}|$ elements to the solution. $\square$

Lemma 4.2.2. The j^{th} pass of Naïve Progressive Greedy as given in Algorithm 4.2 is $(\Delta^{1-j/p}, \Delta^{1/p})$ -bounded for maximum set size Δ .

Proof. If we first consider $j = 1$, no elements have yet been covered, so it is required that all sets are at most of size $\Delta^{1-1/p} \cdot \Delta^{1/p} = \Delta$, which is true by definition of Δ .

With $\tau_j = \Delta^{1-j/p}$ and C_0 as defined in Definition 4.2.1, we now consider larger j . By the logic of Naïve Progressive Greedy, we know that for any set $S \in \mathcal{S}$, if $\text{id}(S)$ exists in $\mathcal{I}$ then S was added in a previous pass and thus contains no uncovered elements. If, on the other hand, the identifier of S is not in $\mathcal{I}$, then S must have contained strictly fewer than τ_{j-1} uncovered elements in the previous pass, otherwise S would have been added to the solution. Thus, as the number of uncovered elements is non-increasing, it follows that $|S \setminus C_0| < \tau_{j-1} = \Delta^{1/p}\tau_j$. $\square$

Lemma 4.2.3. Naïve Progressive Greedy as given in Algorithm 4.2 is a p -pass semi-streaming $(p\Delta^{1/p})$ -approximation algorithm for the SET COVER problem.

Proof. Considering the bounding given in Lemma 4.2.2, it follows from Lemma 4.2.1 that each pass adds at most $\Delta^{1/p} \cdot |\text{OPT}|$ sets to $\mathcal{I}$. Therefore, after p passes, $|\mathcal{I}| \leq p\Delta^{1/p} \cdot |\text{OPT}|$. $\square$

Theorem 4.2.1. Progressive Greedy as given in Algorithm 4.3 is a p -pass semi-streaming $(p+1)\Delta^{1/(p+1)}$ -approximation algorithm for the SET COVER problem.

Proof. Given the same input, it is clear that up until the cover certificate combination step which starts on line 10 of Algorithm 4.3, the values of the common variables held by both p -pass Progressive Greedy and $(p+1)$ -pass Naïve Progressive Greedy are identical. On the next pass, if an uncovered element is encountered, Naïve Progressive Greedy adds to the solution the first set in the input stream which contains this element. Meanwhile, with the secondary cover certificate, Progressive Greedy already knows for each element the set in which it is first encountered. It is clear that, upon merging this secondary cover certificate with the primary one, the result will exactly match that given by Naïve Progressive Greedy. $\square$

4.3 Implementation

For the purposes of an empirical comparison, both MSSSC and Progressive Greedy were implemented in C++. We now detail these implementations.

4.3.1 Implementation of MSSSC

Aided by the decoupled nature of the SSSC implementation, converting SSSC to a subroutine was a simple process, and only a minor modification was required to allow for multiple passes. After each pass, a restricted universe is generated (stored in an `unordered_set`). This is then fed to the SSSC subroutine, which on the next pass disregards all elements not included in this restricted universe, effectively filtering the input sets. Resulting covers are appropriately combined after each pass, and, after all passes have completed, the cover certificate is returned alongside a list of identifiers.

Threshold Value

In practice, we found that using the effectiveness threshold $r_j = (p-j) \lceil \log_2 \Delta \rceil / p$ for the j^{th} pass improves resulting solution sizes, so we use this threshold in our final implementation.

4.3.2 Implementation of Progressive Greedy

We implemented the version of Progressive Greedy given in Algorithm 4.3 (i.e., not the naïve version). So as to calculate a suitable threshold for the first pass, Progressive Greedy requires knowledge of the universe size. To this end, our implementation first does a sweep of the data set to learn this value (though the execution of this code is not included in the total time measurement). Then, subsequent iterations build the solution by maintaining an `unordered_set` to track covered elements. This implementation omits the construction of a cover certificate as such a structure is not required to obtain the desired output format of a list of identifiers (though a temporary cover certificate is still constructed in the final pass to facilitate the folding process).

Threshold Value

Chakrabarti and Wirth use a threshold based on n when presenting the algorithm. In Section 4.2, we apply a minor modification their proof to show that a threshold based on Δ affords an equivalent (if not better, though this depends on the problem instance) approximation factor. With this in mind, to determine whether to base the threshold of our implementation on n or Δ , we applied both such versions of Progressive Greedy to each of the data sets given in Table 3.2 for (arbitrarily) ten passes. Using the threshold based on Δ resulted in a smaller solution size in all but three cases (where the solution sizes were very close), and an average solution size improvement of factor 1.09. Based on this, we proceed with an implementation of Progressive Greedy which uses $\tau_j = \Delta^{1-j/(p+1)}$ as the threshold for the j^{th} pass, where the value Δ is obtained (rather than n) in the initial pass over the stream.

4.4 Empirical Analysis

To determine whether MSSSC performs well in practice, the implementations described in Section 4.3 were applied to each of the data sets in Table 3.2. A tabular summary of the results for all files is given in Appendix B. We only compare solution size, as the time and space consumption of both algorithms are very similar (this is expected, as both process each set in expected linear time and both maintain a cover certificate). Figure 4.3 shows the results for `twitter_graph.dat` as before, which summarises well a pattern that emerges. For comparison with a greedy non-streaming solution, the cover size returned by (RAM-based) DFG with a very low p value is included in the figure. As previously discussed, we know that the solutions produced by DFG are effectively as good as can be approximated in polynomial time, so this serves as a suitable reference.

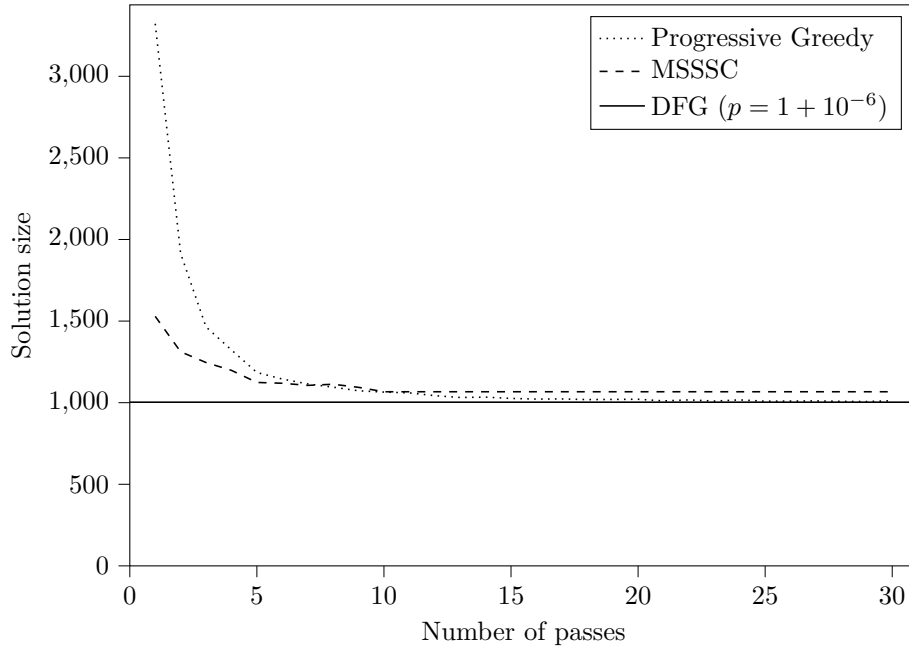


Figure 4.3: Solution size comparison as pass count varies for data file `twitter_graph.dat`.

In a one-pass setting, Progressive greedy is far outperformed by MSSSC (which, in this case, is equivalent to SSSC). For each subsequent pass, the difference between the cover sizes decreases: Progressive Greedy quickly catches up to MSSSC. After a certain point, the solutions given by MSSSC stop improving, while the solution sizes given by Progressive Greedy converge with those given by DFG. This brings to light an inherent weakness of MSSSC: for a given data set, there are at most $\lceil \log_2 \Delta \rceil + 2$ possible effectiveness values (levels), therefore if the universe is partitioned by each of these levels, the performance of the algorithm cannot improve any further. With the effectiveness threshold as formulated in our implementation (i.e., a threshold of $r_j = (p - j) \lceil \log_2 \Delta \rceil / p$ for the j^{th} pass), MSSSC reaches a steady final solution size after $\lceil \log_2 \Delta \rceil + 1$ passes, and any further passes are superfluous.

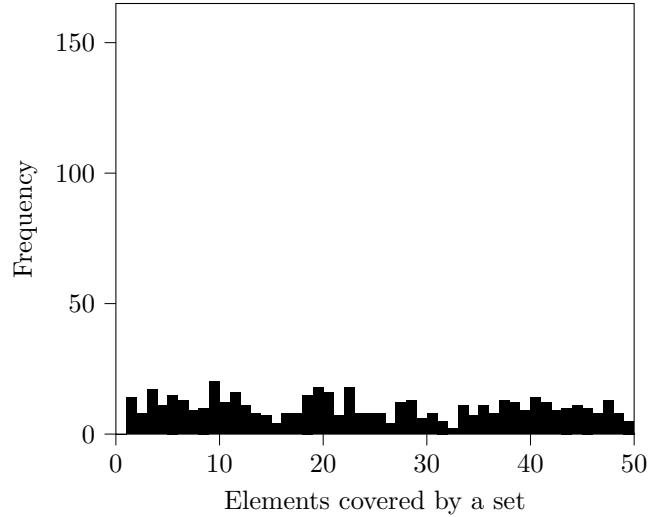


Figure 4.4: Contribution distribution given by 12-pass MSSSC for the file `twitter_graph.dat`.

Figure 4.4 shows the contribution distribution of MSSSC for the file `twitter_graph.dat` after $\lceil \log_2 1205 \rceil + 1 = 12$ passes. By direct comparison with Figure 4.1b, it is clear that the unwanted distribution skew seen in SSSC covers is no longer present; the resulting distribution is now more in line with that given by DFG (shown in Figure 4.1a).

In summary, we have demonstrated that MSSSC is highly capable and performs especially well (significantly better than Progressive Greedy) when few passes over the input stream are made. However, if traversing the input stream in excess of $\lceil \log_2 \Delta \rceil + 1$ times, Progressive Greedy is able to further refine the resulting cover, whereas MSSSC is not.

Chapter 5

Conclusion

In this work, the problem we address is that of approximating SET COVER for very large data sets. Specifically, we investigate whether the Emek-Rosén Semi-Streaming Set Cover (SSSC) algorithm [22] performs well when data set sizes far-exceed the capacity of RAM and the assumption of timely direct access no longer holds. Empirical comparisons with Disk Friendly Greedy (DFG) [16]—an algorithm which addresses this problem but still requires access to an amount of (external) memory linear in size of input—provide a strongly positive answer to this question. We complement these findings with an improved approximation factor for SSSC, stated in terms of the largest set size Δ . As originally presented, SSSC only makes a single pass over the input data set. We generalise SSSC to a new algorithm that makes p passes over the input data set, which we call Multi-pass Semi-Streaming Set Cover (MSSSC). We establish the approximation factor of MSSSC, and compare it both theoretically and empirically to the Progressive Greedy algorithm [12], which executes under the same computational model.

5.1 Contributions

The key objectives set out in Section 1.3 have each been accomplished. We now summarise the main contributions of this work with respect to these original objectives.

SSSC in Practice. We give the results of a comprehensive comparison between DFG and SSSC in Table 3.4, showing that SSSC is highly capable in practice. To acquire these results, we created the first (to the best of our knowledge) implementation of SSSC.

Enhancing Approximation. Though not one of the original objectives, we demonstrate with Algorithm 3.4 (and in our implementation) that it is possible to process each input set in linear time—an improvement of a logarithmic factor. With a modification of the analysis given by Emek and Rosén [22], we also prove in Section 3.6 that SSSC provides an $O(\sqrt{\Delta})$ -approximation to the SET COVER problem, a novel result. We then extend this proof in Section 3.6.1 to give an approximation guarantee in terms of Δ' , a parameter which prevents disproportionately large Δ values severely affecting the approximation factor.

Generalising SSSC. In Chapter 4, we introduce a new algorithm (MSSSC) which generalises SSSC to p passes. We prove by induction in Section 4.1 that MSSSC produces an $O((p+1)\Delta^{1/(p+1)})$ -approximate solution; we show that this approximation factor asymptotically matches that of Progressive Greedy by modifying the original proof of Chakrabarti and Wirth [12] in Section 4.2.1. We then empirically analyse MSSSC, showing that in practice the solution sizes produced are far smaller than those given by Progressive Greedy for low values of p .

To aid future research, we also note that all algorithm implementations described in this work (disk-based DFG, RAM-based DFG, SSSC, Progressive Greedy and MSSSC) will be released as open-source.

5.2 Future Work

We now briefly outline some possible directions in which this work could be taken further.

5.2.1 Other Streaming Algorithms

Considering the practical performance of algorithms is clearly important: often theoretical worst case guarantees are a poor indicator of the typical behaviour of an approximation algorithm for SET COVER [29, 28]. In this work, we implemented and experimented with three semi-streaming algorithms: SSSC, Progressive Greedy, and MSSSC. It would be interesting to perform a broader empirical comparison with other sublinear (i.e., $o(nm)$) space algorithms, including those mentioned in Section 1.2.

5.2.2 SSSC at Web Scale

Both SSSC and MSSSC require $\Theta(n \log m)$ bits of space. For a given problem instance, as long as this space requirement does not exceed the capacity of memory, a solution can be given. For instances where this is not the case, a different approach is required.

The MapReduce paradigm [19, 34] facilitates the processing of data at massive scale by distributing computation between a cohort of machines; we briefly describe it now. A MapReduce iteration is sequential and consists of three stages: *map*, *shuffle*, and *reduce*. In the map phase, each of a series of *mappers* receive as input a key-value pair, and output a collection of key-value pairs without maintaining state. These pairs are then transparently aggregated by key in the shuffle stage, after which the resulting collections of pairs are given to a series of *reducers*. With access to all values associated with a single key, the reducers simultaneously perform computation, and each output a multiset of key-value pairs.

Stergiou and Tsioutsoulis [60] consider the SET COVER problem under this MapReduce paradigm, where their implementation is largely based on a modification of the traditional greedy algorithm.¹ They dedicate one MapReduce iteration to each of a sequence of thresholds, where only those sets with an uncovered element count greater than the threshold of the iteration are processed. During an iteration, the mappers are responsible for this partitioning. Then, the reducers each output a subset of the sets they receive as input, such that all sets they decide not to output contain fewer uncovered elements than the iteration threshold. State is maintained between iterations by keeping a bit vector of size n in shared (distributed) memory, where if the element i is covered, the i^{th} bit of the vector is set.

Having demonstrated in this work that SSSC produces competitive results in practice, it would be interesting to investigate whether the non-greedy nature of SSSC proves beneficial under a paradigm similar to MapReduce, where computation is distributed amongst machines. In particular, could SSSC be used to create a MapReduce algorithm for SET COVER which only requires a single iteration?

5.2.3 Modification to SSSC

SSSC as considered in this work defines the level of a (sub)set as the ceiling of the base 2 logarithm of its size. In aggregating subset sizes with this ceiling function, information is lost. It would be interesting to consider an alternative version of SSSC which avoids this by instead defining the level of a (sub)set as its unchanged size. To preserve the logic of the algorithm, a subset would then be considered effective if its size is at least twice the maximum effectiveness of the elements it contains. This alternative approach is succinctly summarised by considering Definition 5.2.1 and Definition 5.2.2 alongside Algorithm 3.2. A multi-pass generalisation could then be obtained in a manner similar to that described in Section 4.1.

Definition 5.2.1 (Level (modified)). The *level* of some set S is defined as $\text{lev}(S) = |S|$.

Definition 5.2.2 (Effective subset (modified)). A subset $T \subseteq S_t$ is said to be *effective* at time t if for every $x \in T$ it holds that

$$2 \cdot \text{eff}_t(x) \leq \text{lev}(T).$$

Preliminary experimentation with this alternative approach has yielded promising results, and a further (theoretical and practical) investigation would make for an interesting topic of future research.

¹Other closely related problems have also recently been considered under the MapReduce paradigm, see [47, 48, 9, 31] and the references therein.

Bibliography

- [1] BD Acharya. Domination in hypergraphs. *AKCE International Journal of Graphs and Combinatorics*, 4(2):117–126, 2007.
- [2] Pankaj K Agarwal and Jiangwei Pan. Near-linear algorithms for geometric hitting sets and set covers. *Discrete & Computational Geometry*, 63(2):460–482, 2020.
- [3] Noga Alon, Baruch Awerbuch, and Yossi Azar. The online set cover problem. In *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*, pages 100–105, 2003.
- [4] Aris Anagnostopoulos, Luca Becchetti, Ilaria Bordino, Stefano Leonardi, Ida Mele, and Piotr Sankowski. Stochastic query covering for fast approximate document retrieval. *ACM Transactions on Information Systems (TOIS)*, 33(3):1–35, 2015.
- [5] Boris Aronov, Esther Ezra, and Micha Sharir. Small-size ϵ -nets for axis-parallel rectangles and boxes. *SIAM Journal on Computing*, 39(7):3248–3282, 2010.
- [6] Sepehr Assadi. Tight space-approximation tradeoff for the multi-pass streaming set cover problem. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pages 321–335, 2017.
- [7] Sepehr Assadi and Sanjeev Khanna. Tight bounds on the round complexity of the distributed maximum coverage problem. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2412–2431. SIAM, 2018.
- [8] Sepehr Assadi, Sanjeev Khanna, and Yang Li. Tight bounds for single-pass streaming complexity of the set cover problem. *SIAM Journal on Computing*, (0):STOC16–341, 2019.
- [9] MohammadHossein Bateni, Hossein Esfandiari, and Vahab Mirrokni. Distributed coverage maximization via sketching. *arXiv preprint arXiv:1612.02327*, 2016.
- [10] MohammadHossein Bateni, Hossein Esfandiari, and Vahab Mirrokni. Almost optimal streaming algorithms for coverage problems. In *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 13–23, 2017.
- [11] Tom Brijs, Gilbert Swinnen, Koen Vanhoof, and Geert Wets. Using association rules for product assortment decisions: A case study. In *Knowledge Discovery and Data Mining*, pages 254–260, 1999.
- [12] Amit Chakrabarti and Anthony Wirth. Incidence geometries and the pass complexity of semi-streaming set cover. In *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*, pages 1365–1373. SIAM, 2016.
- [13] Tsong Yueh Chen and Man Fai Lau. Dividing strategies for the optimization of a test suite. *Information Processing Letters*, 60(3):135–141, 1996.
- [14] Kenneth L Clarkson and Kasturi Varadarajan. Improved approximation algorithms for geometric set cover. *Discrete & Computational Geometry*, 37(1):43–58, 2007.
- [15] Raffaele Perego Claudio Lucchese, Salvatore Orlando1 and Fabrizio Silvestri. Webdocs: a real-life huge transactional dataset.
- [16] Graham Cormode, Howard Karloff, and Anthony Wirth. Set cover algorithms for very large datasets. In *Proceedings of the 19th ACM international conference on Information and knowledge management*, pages 479–488, 2010.

- [17] Manoranjan Dash. Feature selection via set cover. In *Proceedings 1997 IEEE Knowledge and Data Engineering Exchange Workshop*, pages 165–171. IEEE, 1997.
- [18] Manlio De Domenico, Antonio Lima, Paul Mougél, and Mirco Musolesi. The anatomy of a scientific rumor. *Scientific reports*, 3:2980, 2013.
- [19] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: a flexible data processing tool. *Communications of the ACM*, 53(1):72–77, 2010.
- [20] Erik D Demaine, Piotr Indyk, Sepideh Mahabadi, and Ali Vakilian. On streaming and communication complexity of the set cover problem. In *International Symposium on Distributed Computing*, pages 484–498. Springer, 2014.
- [21] Irit Dinur and David Steurer. Analytical approach to parallel repetition. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, pages 624–633, 2014.
- [22] Yuval Emek and Adi Rosén. Semi-streaming set cover. *ACM Transactions on Algorithms (TALG)*, 13(1):1–22, 2016.
- [23] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM (JACM)*, 45(4):634–652, 1998.
- [24] Joan Feigenbaum, Sampath Kannan, Andrew McGregor, Siddharth Suri, and Jian Zhang. On graph problems in a semi-streaming model. *Departmental Papers (CIS)*, page 236, 2005.
- [25] Karolien Geurts, Geert Wets, Tom Brijs, and Koen Vanhoof. Profiling high frequency accident locations using association rules. In *Proceedings of the 82nd Annual Transportation Research Board, Washington DC. (USA), January 12-16*, page 18pp, 2003.
- [26] Bart Goethals. Frequent itemset mining dataset repository.
- [27] Lukasz Golab, Howard Karloff, Flip Korn, Divesh Srivastava, and Bei Yu. On generating near-optimal tableaux for conditional functional dependencies. *Proceedings of the VLDB Endowment*, 1(1):376–390, 2008.
- [28] Fernando C Gomes, Cláudio N Meneses, Panos M Pardalos, and Gerardo Valdisio R Viana. Experimental analysis of approximation algorithms for the vertex cover and set covering problems. *Computers & Operations Research*, 33(12):3520–3534, 2006.
- [29] Tal Grossman and Avishai Wool. Computational experience with approximation algorithms for the set covering problem. *European Journal of Operational Research*, 101(1):81–92, 1997.
- [30] Sarel Har-Peled, Piotr Indyk, Sepideh Mahabadi, and Ali Vakilian. Towards tight bounds for the streaming set cover problem. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pages 371–383, 2016.
- [31] Nicholas JA Harvey, Christopher Liaw, and Paul Liu. Greedy and local ratio algorithms in the mapreduce model. In *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures*, pages 43–52, 2018.
- [32] Chengbang Huang, Faruck Morcos, Simon P Kanaan, Stefan Wuchty, Danny Z Chen, and Jesus A Izaguirre. Predicting protein-protein interactions from protein domains using a set cover approach. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 4(1):78–87, 2007.
- [33] Viggo Kann. *On the approximability of NP-complete optimization problems*. PhD thesis, Royal Institute of Technology Stockholm, 1992.
- [34] Howard Karloff, Siddharth Suri, and Sergei Vassilvitskii. A model of computation for mapreduce. In *Proceedings of the twenty-first annual ACM-SIAM symposium on Discrete Algorithms*, pages 938–948. SIAM, 2010.
- [35] Richard M Karp. Reducibility among combinatorial problems. In *Complexity of computer computations*, pages 85–103. Springer, 1972.

- [36] Christine Klymko, David Gleich, and Tamara G Kolda. Using triangles to improve community detection in directed networks. *arXiv preprint arXiv:1404.5874*, 2014.
- [37] Fabian Kuhn, Pascal Von Rickenbach, Roger Wattenhofer, Emo Welzl, and Aaron Zollinger. Interference in cellular networks: The minimum membership set cover problem. In *International Computing and Combinatorics Conference*, pages 188–198. Springer, 2005.
- [38] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. What is twitter, a social network or a news media? In *Proceedings of the 19th international conference on World wide web*, pages 591–600, 2010.
- [39] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. Predicting positive and negative links in online social networks. In *Proceedings of the 19th international conference on World wide web*, pages 641–650, 2010.
- [40] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. Signed networks in social media. In *Proceedings of the SIGCHI conference on human factors in computing systems*, pages 1361–1370, 2010.
- [41] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>, June 2014.
- [42] Jure Leskovec, Kevin J. Lang, Anirban Dasgupta, and Michael W. Mahoney. Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters. *CoRR*, abs/0810.1355, 2008.
- [43] Jure Leskovec and Julian J Mcauley. Learning to discover social circles in ego networks. In *Advances in neural information processing systems*, pages 539–547, 2012.
- [44] Hyojun Lim and Chongkwon Kim. Flooding in wireless ad hoc networks. *Computer Communications*, 24(3-4):353–363, 2001.
- [45] Wei Lou and Jie Wu. On reducing broadcast redundancy in ad hoc wireless networks. In *36th Annual Hawaii International Conference on System Sciences, 2003. Proceedings of the*, pages 10–pp. IEEE, 2003.
- [46] Robert Meusel, Sebastiano Vigna, Oliver Lehmberg, and Christian Bizer. Graph structure in the web—revisited: a trick of the heavy tail. In *WWW Companion*, pages 427–432, 2014.
- [47] Baharan Mirzasoleiman, Amin Karbasi, Ashwinkumar Badanidiyuru, and Andreas Krause. Distributed submodular cover: Succinctly summarizing massive data. In *Advances in Neural Information Processing Systems*, pages 2881–2889, 2015.
- [48] Baharan Mirzasoleiman, Morteza Zadimoghaddam, and Amin Karbasi. Fast distributed submodular cover: Public-private data summarization. In *Advances in Neural Information Processing Systems*, pages 3594–3602, 2016.
- [49] Alan Mislove, Massimiliano Marcon, Krishna P. Gummadi, Peter Druschel, and Bobby Bhattacharjee. Measurement and Analysis of Online Social Networks. In *IMC*, San Diego, CA, October 2007.
- [50] Kamesh Munagala, Shivnath Babu, Rajeev Motwani, and Jennifer Widom. The pipelined set cover problem. In *International Conference on Database Theory*, pages 83–98. Springer, 2005.
- [51] Nabil H Mustafa, Rajiv Raman, and Saurabh Ray. Settling the apx-hardness status for geometric set cover. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pages 541–550. IEEE, 2014.
- [52] Noam Nisan. The communication complexity of approximate set packing and covering. In *International Colloquium on Automata, Languages, and Programming*, pages 868–875. Springer, 2002.
- [53] Jeff Offutt, Jie Pan, and Jeffrey M Voas. Procedures for reducing the size of coverage-based test sets. In *Proceedings of the 12th International Conference on Testing Computer Software*, pages 111–123. ACM Press New York, 1995.
- [54] David Reinsel, John Gantz, and John Rydning. The digitization of the world from edge to core. *IDC White Paper*, 2018.

- [55] Wei Ren, Qing Zhao, Ram Ramanathan, Jianhang Gao, Ananthram Swami, Amotz Bar-Noy, Matthew P Johnson, and Prithwish Basu. Broadcasting in multi-radio multi-channel wireless networks using simplicial complexes. *Wireless networks*, 19(6):1121–1133, 2013.
- [56] Matei Ripeanu, Ian Foster, and Adriana Iamnitchi. Mapping the gnutella network: Properties of large-scale peer-to-peer systems and implications for system design. *arXiv preprint cs/0209028*, 2002.
- [57] Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization. In *AAAI*, 2015.
- [58] Barna Saha and Lise Getoor. On maximum coverage in the streaming model & application to multi-topic blog-watch. In *Proceedings of the 2009 siam international conference on data mining*, pages 697–708. SIAM, 2009.
- [59] Petr Slavík. A tight analysis of the greedy algorithm for set cover. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 435–441, 1996.
- [60] Stergios Stergiou and Kostas Tsioutsoulouklis. Set cover at web scale. In *Proceedings of the 21th acm sigkdd international conference on knowledge discovery and data mining*, pages 1125–1133, 2015.
- [61] Zheng Wu, Thomas H Kunz, and Margrit Betke. Efficient track linking methods for track graphs using network-flow and set-cover techniques. In *CVPR 2011*, pages 1185–1192. IEEE, 2011.
- [62] Jaewon Yang and Jure Leskovec. Defining and evaluating network communities based on ground-truth. *CoRR*, abs/1205.6233, 2012.
- [63] Hao Yin, Austin R Benson, Jure Leskovec, and David F Gleich. Local higher-order graph clustering. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 555–564, 2017.
- [64] Shin Yoo. A novel mask-coding representation for set cover problems with applications in test suite minimisation. In *2nd International Symposium on Search Based Software Engineering*, pages 19–28. IEEE, 2010.

Appendix A

Set Size Distribution

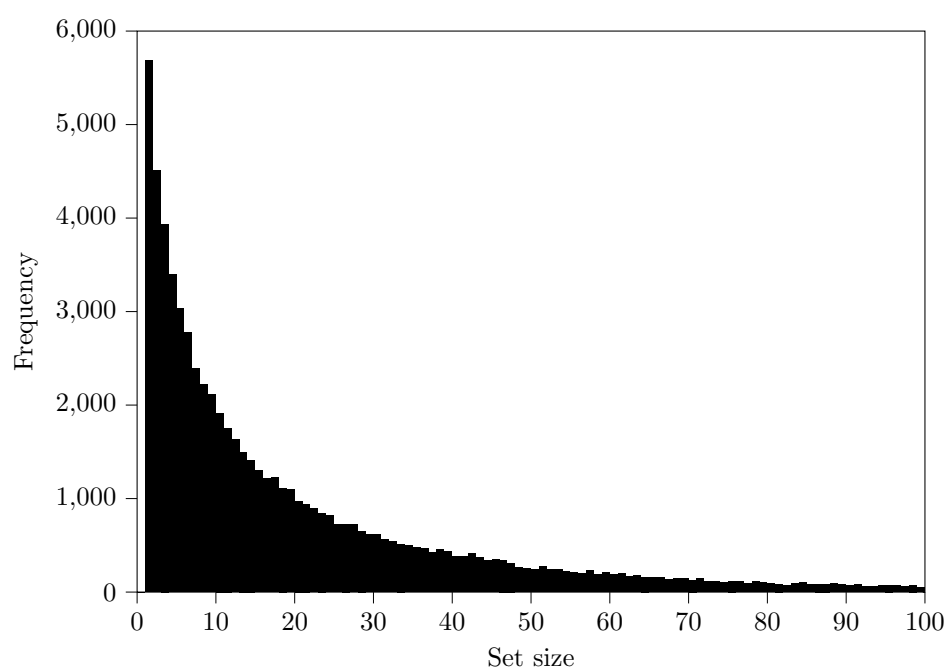


Figure A.1: Set size distribution for the file `twitter_graph.dat`.

Appendix B

Multi-Pass Solution Quality

File name	Passes					
	1	2	4	8	16	32
chess	13	12	11	10	10	10
mushroom	27	26	25	24	24	24
accidents	213	200	193	181	181	181
pumsb_star	904	867	819	774	774	774
pumsb	904	867	817	774	774	774
wikivote_graph	236	202	181	175	176	176
retail	5908	5590	5329	5159	5159	5159
kosarak	18618	18241	17976	17809	17773	17773
youtube.cmt	9497	9482	9443	9421	9412	9412
gnutella_graph	13839	13689	13636	13491	13491	13491
twitter_graph	1529	1312	1198	1113	1067	1067
dblp.cmt	11409	11321	11306	11302	11298	11298
dblp_ungraph	59541	55342	53061	51679	51680	51680
amazon.cmt	3851	3791	3783	3779	3774	3774
amazon_ungraph	77893	76254	70750	65837	64973	64973
higgs_twitter_graph	90555	86969	83559	81786	81177	81177
berk_stan_graph	140399	138192	136494	134259	134259	134259
google_graph	161983	158360	155514	153006	153005	153005
lj.cmt	153321	150762	148965	148348	148071	148071
youtube_ungraph	238619	236067	233568	232453	231878	231878
wikitc.cmt	14496	14338	14233	14132	14108	14105
wikitc_graph	295132	283525	272851	268661	265526	265526
ca_road_graph	833385	828561	802312	802312	802312	802312
orkut.cmt	158439	155803	152518	150597	149686	149686
wikitalk_graph	32890	32815	32809	32807	32807	32807
orkut_ungraph	229409	194459	155606	126734	113286	113286
lj_ungraph	941801	910369	850102	820634	802793	802793
lj1_graph	1096844	1070196	1009242	981961	965319	965319
webdocs	413819	412046	409531	408147	407391	407391
friendster.cmt	781962	776729	772009	770719	770033	770033
orkut_u2g_graph	994050	988376	954737	931265	919342	919342
twitter	4284164	4103016	4006699	3919679	3874716	3858555
web_pld_graph	4328159	4269311	4210309	4166910	4147878	4141364
friendster_ungraph	12434527	11758867	11210877	10982763	10820970	10820970
flsd_graph	9593175	9489921	9368704	9251389	9163737	9143867

Table B.1: Solution sizes given by p -pass MSSSC as p increases in powers of 2.

File name	Passes					
	1	2	4	8	16	32
chess	15	13	11	9	9	9
mushroom	32	26	24	23	22	22
accidents	271	220	204	181	183	180
pumsb_star	1223	1031	905	768	756	749
pumsb	1246	1079	898	764	754	749
wikivote_graph	317	245	207	192	174	175
retail	7870	6736	5805	5142	5112	5108
kosarak	22931	21673	20491	19594	17752	17747
youtube.cmt	10316	10165	9867	9613	9411	9401
gnutella_graph	14590	14102	13717	13501	13462	13428
twitter_graph	3322	1914	1323	1095	1022	1003
dblp.cmt	11557	11523	11430	11343	11298	11288
dblp_ungraph	111891	76584	63744	52965	50842	50380
amazon.cmt	5072	4760	4334	4007	3767	3652
amazon_ungraph	136811	102262	80840	71570	63400	63001
higgs_twitter_graph	125332	110659	97972	88340	80875	80763
berkstan_graph	198473	181203	167603	135628	132027	131102
google_graph	223277	191011	171597	155118	151064	148573
lj.cmt	211434	183149	167472	156693	153265	147791
youtube_ungraph	375042	332200	288703	258898	232601	231672
wikitc.cmt	15511	15082	14540	14289	14169	14092
wikitc_graph	457323	384967	329352	288734	265352	264427
ca_road_graph	1136154	884546	802312	789027	788232	788232
orkut.cmt	277714	221906	190027	163721	149697	149388
wikitalk_graph	36452	34069	33391	33014	32808	32807
orkut_ungraph	570447	345103	213759	146985	115368	108615
lj_ungraph	1891558	1451101	1077539	882821	809053	795245
lj1_graph	2114665	1674708	1299751	1113828	972786	958245
webdocs	492557	467305	442794	424382	408317	406867
friendster.cmt	935288	877983	822873	786147	770384	769492
orkut_u2g_graph	1287838	1223221	1094908	993287	923846	916538
twitter	6938439	6280132	5471353	4745332	4189891	3853205
web_pld_graph	5889316	5506438	5082900	4669326	4350261	4133966
friendster_ungraph	20501351	16668975	13337954	11591056	10807425	10770504
flsd_graph	12663744	11971312	11077635	10119967	9564862	9109225

Table B.2: Solution sizes given by p -pass Progressive Greedy as p increases in powers of 2.